COLLEGE OF BUSINESS AND ECONOMICS

SCHOOL OF BUSINESS

BUSINESS INFORMATION TECHNOLOGY

YEAR IV

MODULE TITLE: BUSINESS INTELLIGENCE

LECTURE: RUKUNDO Prince

Analytical Assignment

PREPARED BY: UMUTONI HIDAYA

REGISTRATION NUMBER: 222020222

Date of submission: Sunday, September 28, 2026

Q1. Does High Revenue Mean High Profit?

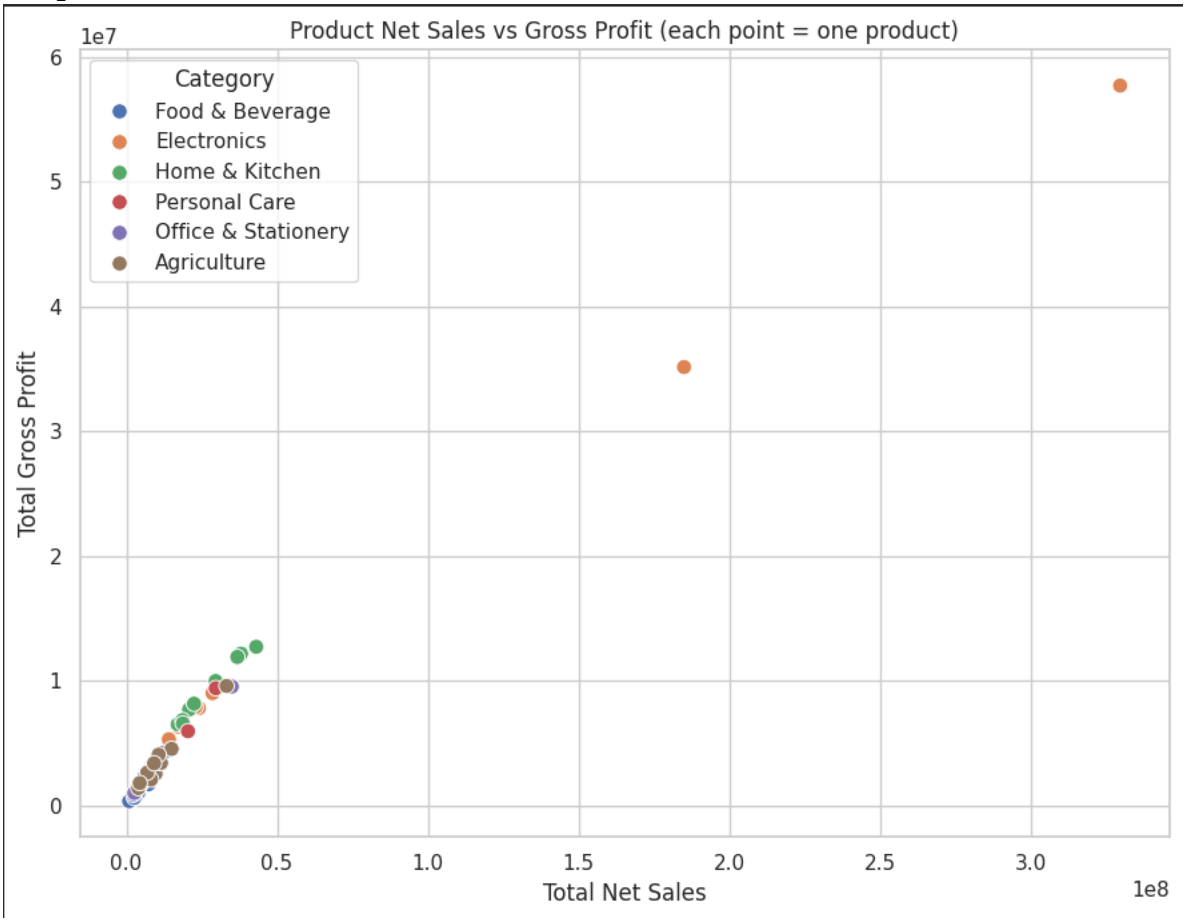
Answer: No.

Expected finding: The product with the highest Net Sales may not have the highest Gross Profit.

Why: Products have different Unit Costs and profit margins.

Two products with similar sales can generate different profits because their costs are different.

Output



Interpretation: At the very top of the list, the assumption largely holds: the two Smartphone products and the leading Home & Kitchen items rank #1–#5 in both sales and profit, so the biggest revenue generators are, in absolute terms, also the biggest profit generators.

However, the assumption breaks down once we move past the top few products. Staple, low-margin items such as Bean Seed 2kg, Maize Seed 2kg (Agriculture) and Rice 5kg, Cooking Oil 1L, Tea 100 Bags (Food & Beverage) sell in decent volumes (mid-table on the sales ranking) but fall several places further down the profit ranking — their profit margins (23%–26%) are noticeably thinner. Conversely, items such as Hand Hoe, Water Bottle, Stapler and Electric

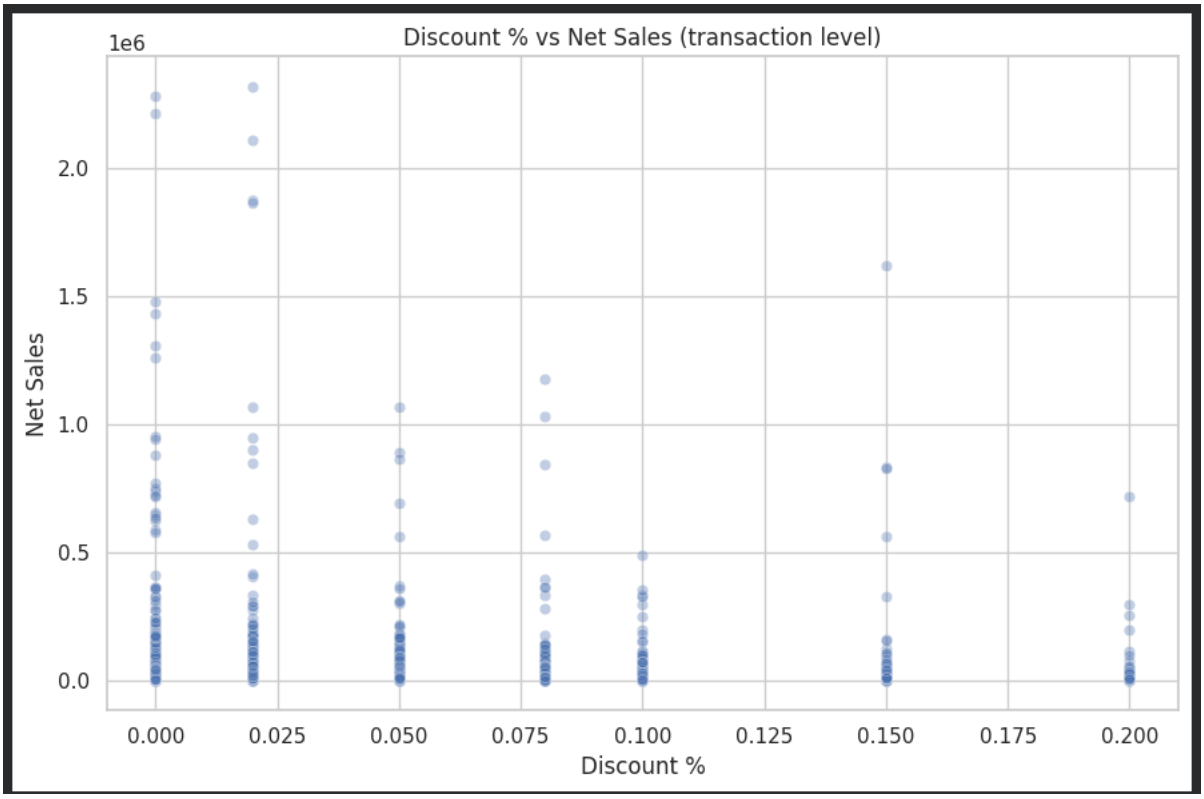
Kettle sell relatively modestly but carry margins of 34%–40%, so they climb several places when ranked by profit instead of sales.

Why can two products with similar sales generate very different profit? Because GrossProfit is driven by the margin (selling price vs. unit cost), not by revenue itself. Two products can generate identical Net Sales while one has a unit cost close to its selling price (thin margin, e.g. electronics where components are expensive) and the other has a unit cost far below its selling price (fat margin, e.g. some home & kitchen or hardware items).

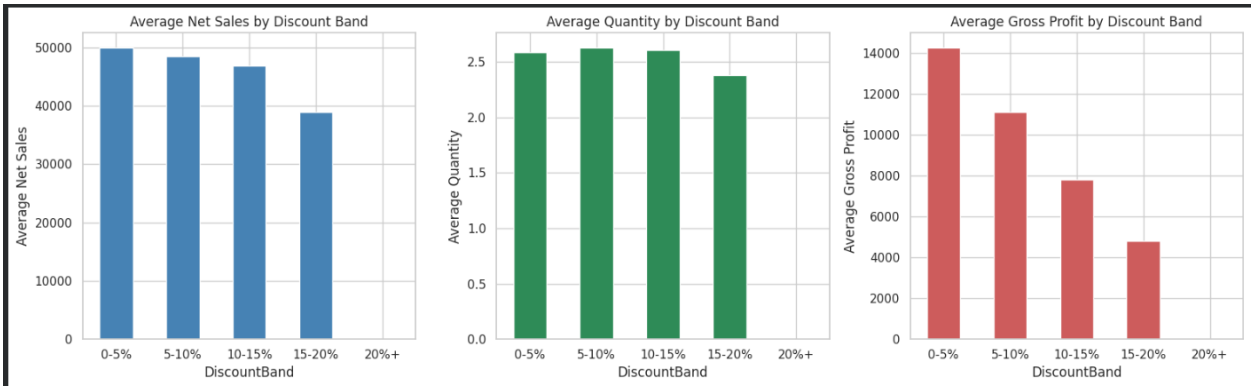
Management implication: Net Sales alone is not a safe way to prioritise products for promotion, shelf space, or marketing spend. Products should be evaluated on both sales volume and profit margin — a high-volume, low-margin staple and a lower-volume, high-margin item can be equally valuable to the business, but for different reasons (footfall/volume vs. profitability).

Q2. Are Discounts Associated With Sales Performance?

Answer: Yes, potentially.
Expected finding: Higher discounts may be associated with higher Quantity or Net Sales, but lower Gross Profit.
Why: Discounts can encourage customers to buy more, while reducing the amount earned per product.
The analysis can show association, not causation.
Output i



Output ii



Interpretation

The correlation between DiscountPct and NetSales is essentially zero (around -0.02), and the correlation with Quantity is also essentially zero (around 0.0004). In other words, transactions with bigger discounts do not, on average, involve buying more units or generating more revenue. The band comparison confirms this: average Net Sales actually falls as the discount band increases (from about 50,000 in the 0–5% band down to about 39,000 in the 15–20% band), average Quantity per transaction stays essentially flat across all bands (~2.4–2.6 units), and average Gross Profit drops sharply as discount increases (from about 14,300 down to about 4,800) — which is expected, since a larger discount directly reduces the revenue collected per unit sold.

What the data allows us to conclude: In this dataset, higher discount levels are associated with lower average Net Sales and substantially lower average Gross Profit per transaction, and are not associated with any increase in units purchased. There is no evidence in this dataset that discounting increases how much customers buy.

What the data does NOT allow us to conclude: We cannot conclude that discounts cause lower sales, or that removing discounts would increase sales. The discount level in this data was not randomly assigned as part of a controlled experiment — it may already reflect other factors we have not controlled for, such as the product category, the customer type receiving negotiated pricing, or the sales channel. This is a classic case of correlation, not causation: a lower Net Sales value on discounted transactions could reflect why a discount was given (e.g. smaller, price-sensitive orders) rather than the discount itself causing the lower value. To test causation properly, management would need a controlled experiment (e.g. a randomised trial offering different discount levels to comparable customers).

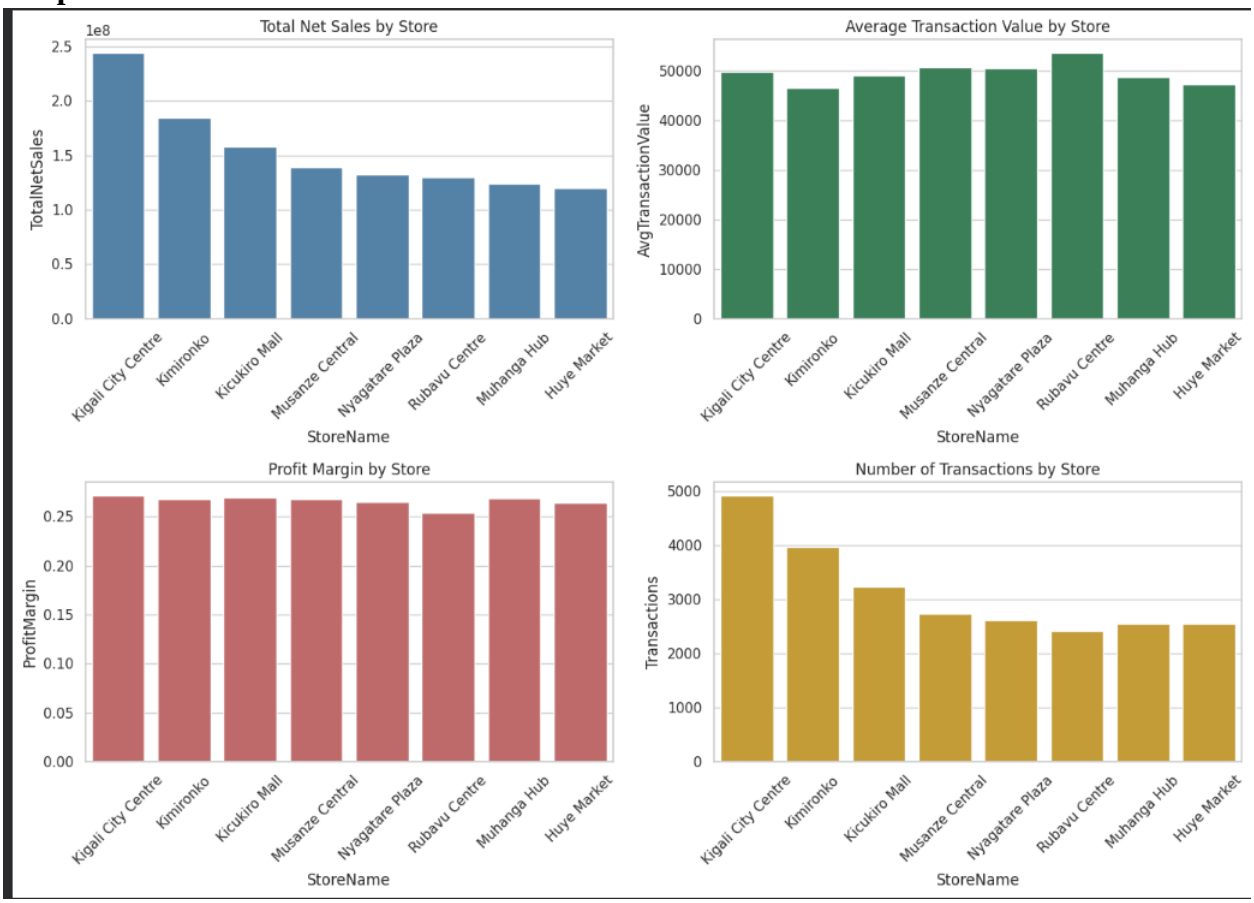
Management implication: the dataset gives no evidence to support the idea that "larger discounts will increase sales" — if anything, larger discounts here are associated with weaker transactions

and clearly erode profit. Before rolling out bigger discounts, management should test the idea in a controlled pilot rather than assume it will work.

Q3. Which Store Is Really Performing Best?

Answer: The best store should be selected using several indicators, not Net Sales alone.
Expected finding: The highest-sales store may not have the highest Gross Profit or Profit Margin.
Why: Stores can have different transaction volumes, quantities, and profit margins.
Key measures: Net Sales, Gross Profit, Profit Margin, Average Transaction Value, and transactions.

Output



Interpretation

Kigali City Centre leads on Net Sales, transaction count, units sold and gross profit, and also has the highest profit margin (about 27.2%) among all eight stores — so in this dataset it is genuinely the strongest store overall, not just the biggest by revenue.

But the ranking is not consistent across every measure for the other stores, which is exactly the warning the assignment asks us to investigate. Rubavu Centre ranks only 6th by Total Net Sales, yet it has the 2nd-highest Average Transaction Value of all stores (≈53,500, second only overall)

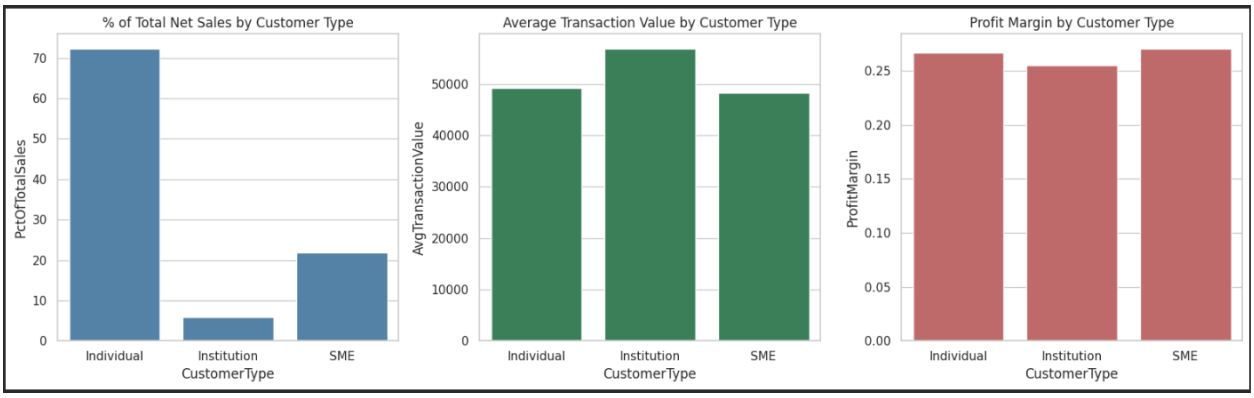
— its customers buy in relatively large baskets. Despite this, Rubavu Centre has the lowest profit margin of all 8 stores (≈25.4%, versus 26.4%–27.2% elsewhere). A manager looking only at "average transaction value" or "Net Sales" would rate Rubavu Centre as a healthy, high-value store, while in fact it is converting that revenue into profit less efficiently than every other store.

Why might the highest-Net-Sales store not be the strongest store overall? Net Sales combines two very different things — how many transactions a store processes and how large and profitable each transaction is — into a single number. A store can reach high Net Sales through sheer transaction volume while carrying a lower margin, or through fewer but larger transactions that still erode profit if the cost base is higher (e.g. different product mix, more discounting, different customer types). Only by looking at Transactions, Average Transaction Value and Profit Margin side by side can management see which stores are efficient, which are large-but-thin-margin, and which genuinely combine scale with profitability.

Q4. Which Customer Type Creates the Most Business Value?

Answer: The customer type with the strongest overall combination of sales and profitability.
Expected finding: The highest-sales customer type may not be the most profitable.
Why: Individual, Institution, and SME customers may differ in transaction size, quantity, and profit margin.
The analysis should compare all three types before selecting the strongest one.

Output



Interpretation

Individual customers dominate the business: 3,600 of the roughly 4,975 customers (about 72%), about 18,100 transactions, and about 72% of total Net Sales. SMEs contribute a solid second place (about 22% of sales from only 1,119 customers), while Institutions are a small group (256 customers) but contribute meaningfully (≈6% of sales) with by far the highest average transaction value (≈56,900 vs. ≈48,000–49,000 for the other two types) — consistent with institutions placing fewer, larger, bulk orders.

On profitability, the picture changes: SME customers have the highest profit margin (≈27.1%), Individuals sit in the middle (≈26.7%), and Institutions have the lowest margin (≈25.5%) despite their large average order size.

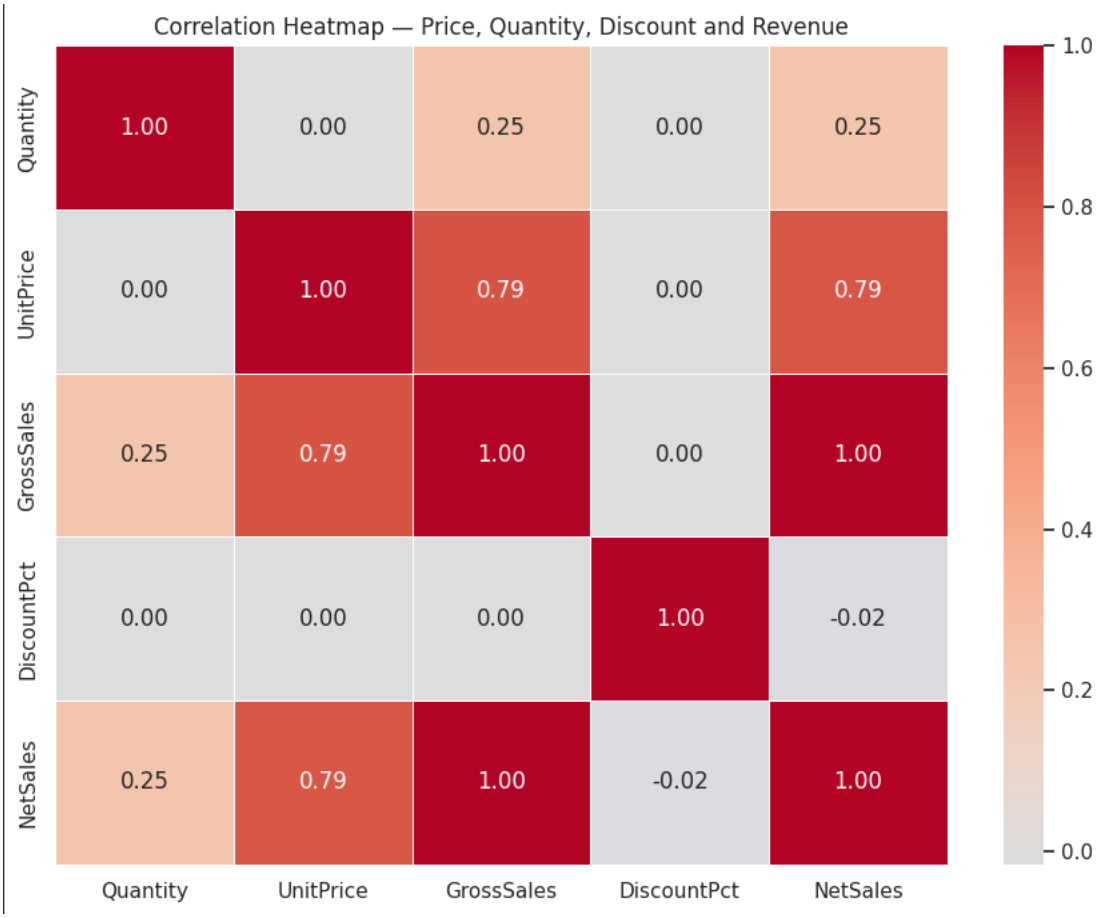
Is the customer type with the highest sales also the most profitable? No. Individuals generate by far the most sales (72%) but are not the most profitable customer type on a margin basis — SMEs are. Institutions generate large average transactions but convert them into profit least efficiently, likely because bulk/negotiated pricing compresses their margin.

Management implication: the business should not treat "customer type with the biggest revenue share" and "customer type management should prioritise for profitability" as the same thing. Individuals remain the core volume driver and should not be neglected, but SMEs deserve attention as the most profit-efficient segment, and any further discounting or bulk-pricing policy for Institutional accounts should be reviewed carefully, since it is already the lowest-margin segment.

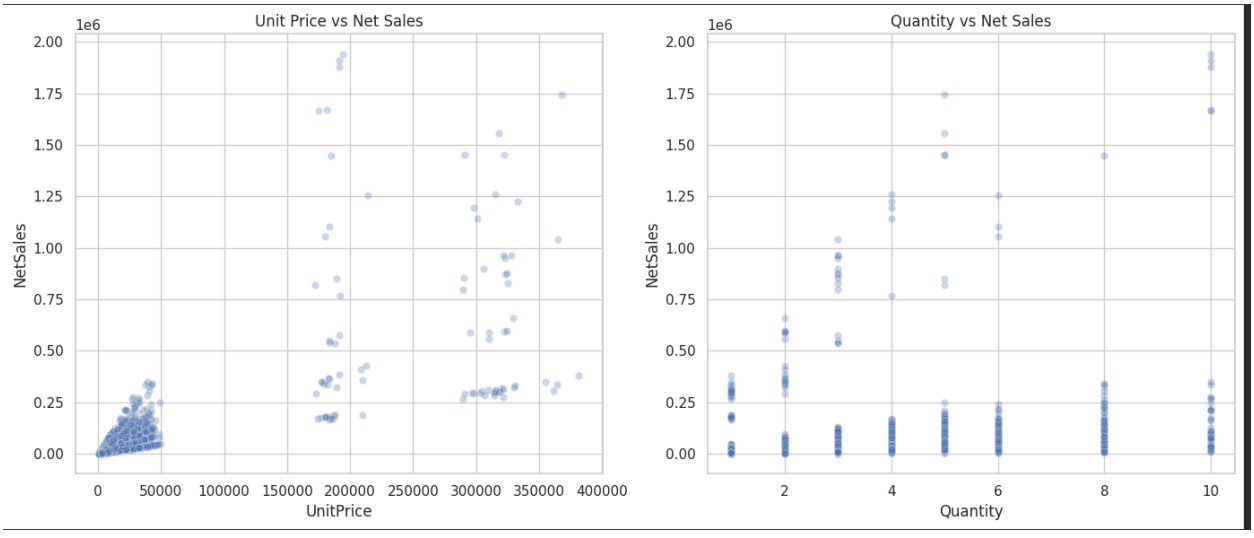
Q5. What Is the Relationship Between Product Price, Quantity and Revenue?

> **Answer**: Strong relationships are expected between Quantity, UnitPrice, and NetSales.
> Expected finding: Quantity and UnitPrice will likely have strong relationships with NetSales.
> Why: Sales value is mathematically based on price and quantity.
> Therefore, some correlations are expected from the formula and are not necessarily new business insights.

Output i



Output ii



Interpretation

NetSales correlates almost perfectly with GrossSales (≈0.999). UnitPrice correlates strongly with NetSales (≈0.79), while Quantity correlates only moderately (≈0.25), and DiscountPct correlates essentially not at all with NetSales (≈-0.02).

Which of these are genuine business insights, and which are data-generated artefacts of the formulas?

GrossSales ↔ NetSales (≈0.999) and UnitPrice ↔ GrossSales (≈0.79) are not real discoveries — GrossSales = Quantity × UnitPrice and NetSales ≈ GrossSales × (1 − DiscountPct) by construction. A near-perfect correlation here simply confirms the formula was applied correctly; it tells management nothing new about customer behaviour.

The comparison between UnitPrice (≈0.79) and Quantity (≈0.25) as drivers of NetSales is a genuine, useful business insight: transaction value in this business is driven far more by which, higher-priced product is bought than by how many units are bought in one visit. This tells management that revenue growth is more likely to come from selling higher-value products (upselling, product mix) than from encouraging customers to buy more units of the same item.

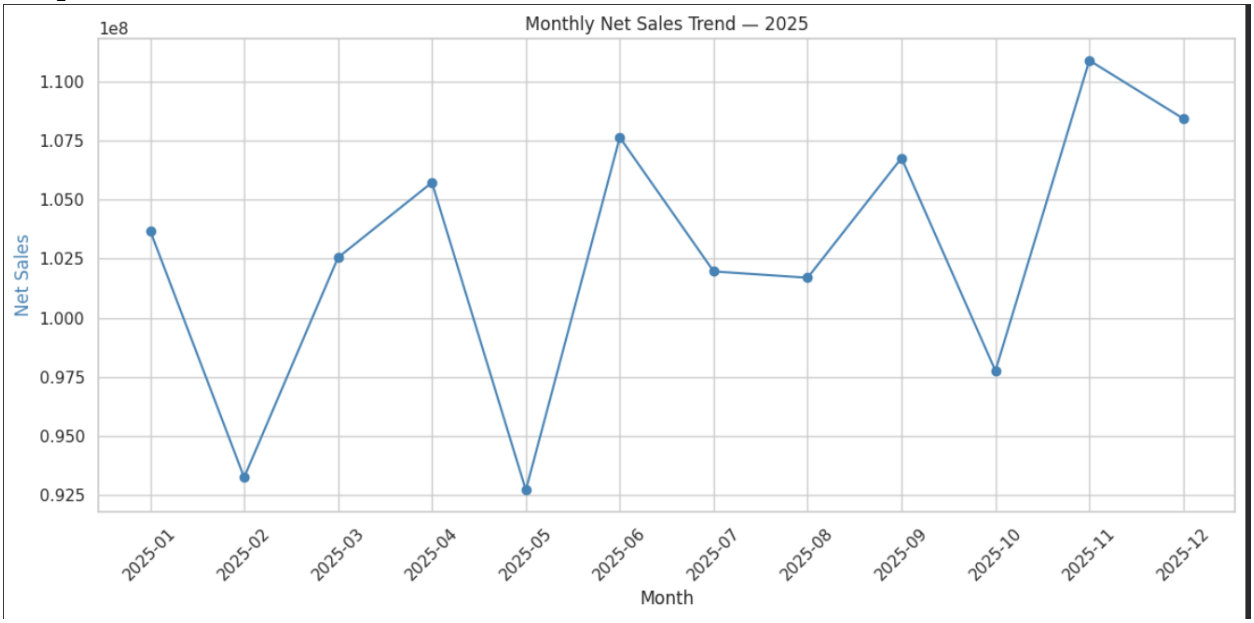
The very weak DiscountPct ↔ NetSales correlation (≈-0.02) reinforces the Question 2 finding: in this dataset, discounting is not a meaningful lever for either raising or lowering the size of a transaction once price and quantity are already accounted for.

Management implication: focus merchandising and sales effort on product mix and pricing (which move the needle), rather than assuming basket size (Quantity) or discounting are the main levers of revenue — neither shows a strong relationship with NetSales in this dataset.

Q6. Does the Company Have a Monthly Sales Pattern?

> **Answer**: Yes, if the data shows consistent changes over time.
> Expected finding: Some months will have higher or lower Net Sales than others.
> Why: Customer demand, transaction numbers, and quantities sold can change from month to month.
> A repeated pattern at similar times would provide stronger evidence of seasonality.

Output



Interpretation

Net Sales moves up and down noticeably from month to month (roughly between 93 million and 111 million), with no single month standing far apart from the rest. Quarterly totals show a mild, fairly steady upward drift across the year (Q1 lowest, Q4 highest, an increase of roughly 6% from Q1 to Q4), rather than one dramatic jump.

What evidence suggests performance changes over time? Two things: (1) the month-to-month percentage changes swing by double digits in both directions (the largest single-month jump and the largest single-month drop are both more than 10%), showing real volatility rather than a flat line; and (2) the quarter-by-quarter totals rise consistently from Q1 through Q4, which is a genuine, measurable trend within this year's data.

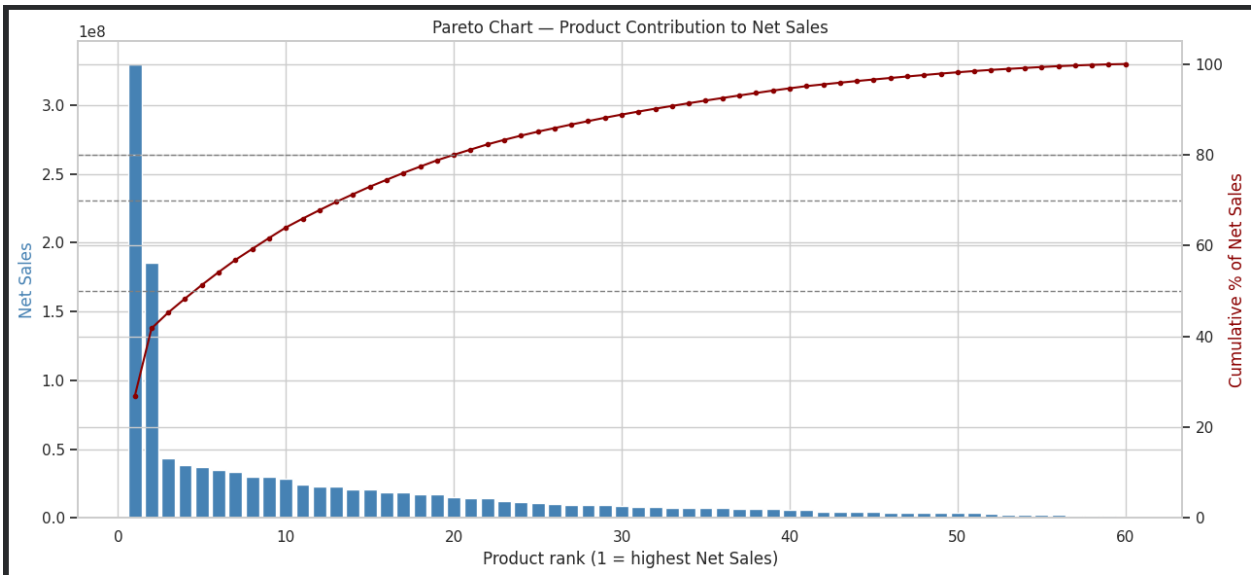
Why we do not call this "seasonality" yet: seasonality means a pattern that repeats in the same calendar months every year (e.g. always higher in December). With only twelve months of data from a single year, we cannot distinguish a true recurring seasonal effect from a one-off trend, random month-to-month noise, or a business-specific event (a promotion, a new store opening, stock availability) that happened to occur in a particular month. Confirming seasonality would require at least two or three years of comparable monthly data showing the same months consistently outperforming or underperforming.

Management implication: treat the observed monthly swings as evidence that performance is not flat, and keep tracking month-to-month change as a KPI, but avoid labelling any specific month as "seasonally strong/weak" or building inventory and staffing plans around an assumed yearly cycle until more years of data confirm the pattern repeats.

Q7. Do a Small Number of Products Generate Most Revenue?

Answer: Yes, if the cumulative analysis shows high concentration.
Expected finding: A relatively small number of products may generate a large percentage of total sales.
Why: Some products may have much higher demand than others.
The Pareto analysis will show how many products generate 50%, 70%, and 80% of revenue.

Output



Interpretation

Revenue is highly concentrated: only about 5 of the 60 products (roughly 8%) are needed to reach 50% of total Net Sales, about 14 products (roughly 23%) reach 70%, and about 21 products (roughly 35%) reach 80%. In other words, roughly one-third of the catalogue generates four-fifths of total revenue, while the remaining two-thirds of products together contribute only about a fifth of sales. This concentration is largely driven by a small number of high-value Electronics items (notably the two smartphone products), which alone make up a very large share of total revenue.

Is revenue highly concentrated among a small group of products? Yes.

Implications for inventory and product management:

The small set of "star" products deserves priority in stock availability — running out of a top-5 product has an outsized impact on total revenue, so safety-stock levels and reorder frequency for these items should be higher than for the long tail.

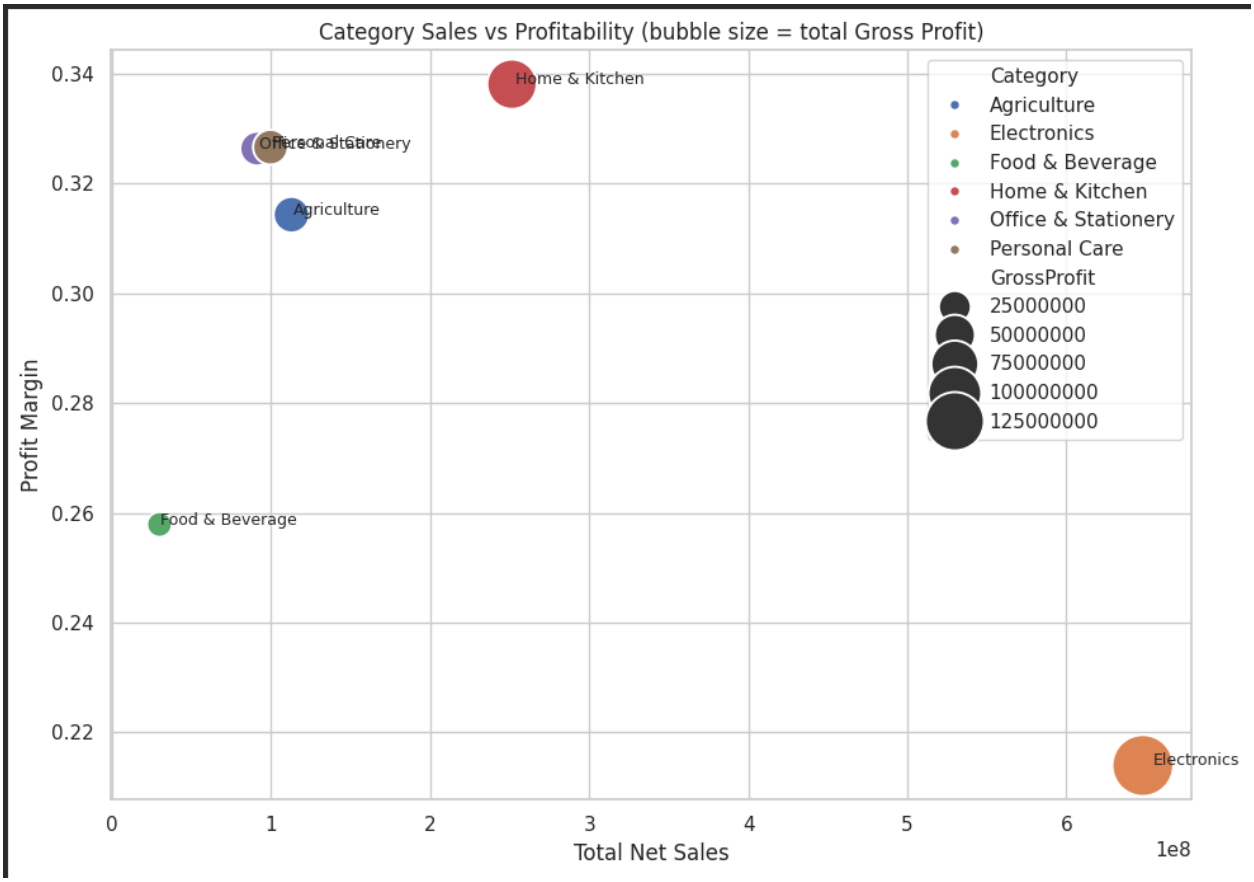
The large "long tail" of low-contribution products should be reviewed for rationalisation: some may be candidates for reduced shelf space, bundling, or discontinuation if they tie up inventory capital and staff time without contributing meaningfully to revenue — although Question 1's findings show some of these lower-revenue items carry strong margins, so profitability (not just sales) should be checked before removing any product.

Because revenue depends so heavily on a handful of SKUs, the business also carries concentration risk — a supply disruption or a competitor undercutting price on just one or two top products could materially affect total company revenue.

Q8. Which Product Categories Are Driving the Business?

Answer: The categories with the highest combination of Net Sales and Gross Profit.
Expected finding: Some categories will contribute more sales and profit than others.
Why: Categories differ in demand, prices, costs, and product mix.
A category can have high sales but a relatively low margin, so both sales and profitability must be considered.

Output



Interpretation

Electronics — high sales, relatively low margin. Electronics contributes about 53% of total Net Sales, by far the largest share, but its profit margin (≈21%) is the lowest of all six categories — a direct echo of Question 1's finding about the two smartphone products.

Home & Kitchen — high sales AND high margin. Home & Kitchen is the second-biggest category (≈20% of sales) and also carries the highest margin of all categories (≈34%) — currently the strongest combination of scale and profitability in the business.

Agriculture, Personal Care, Office & Stationery — lower sales, relatively high margin. Each contributes only about 7–9% of total sales individually, but all three carry healthy margins in the 31%–33% range.

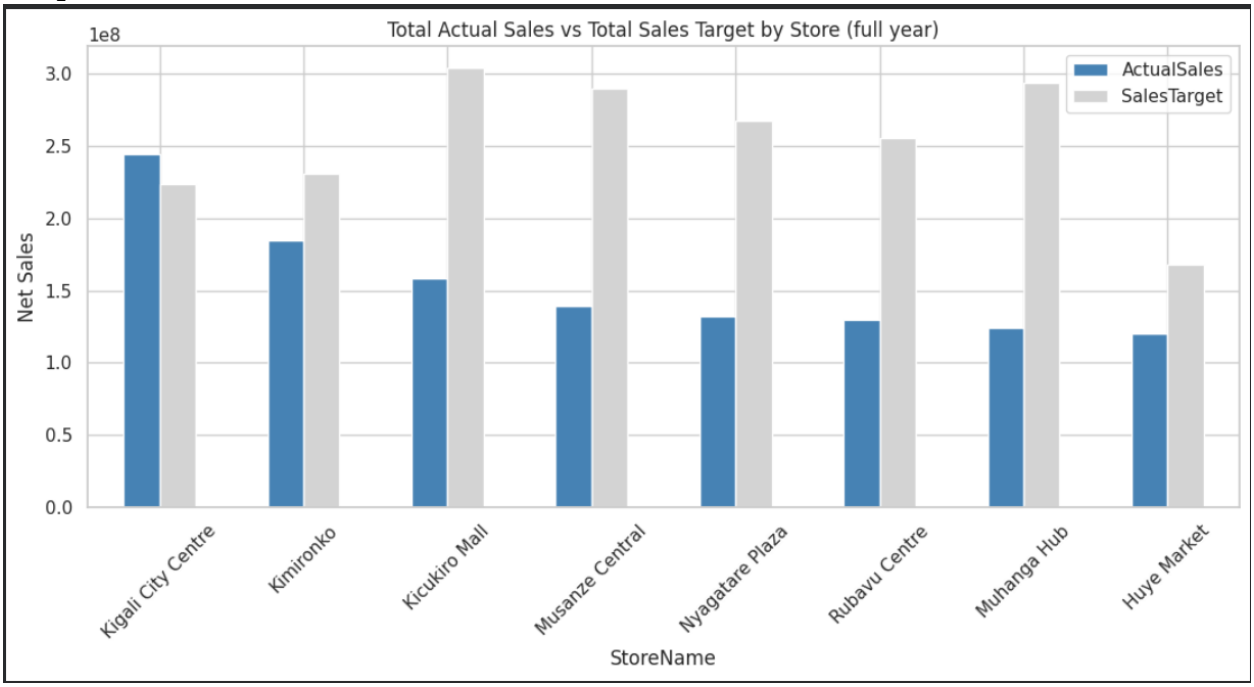
Food & Beverage — lowest sales, and a below-average margin. This category contributes only about 2.4% of total sales, the smallest of all six, and its margin (≈26%) is also below every category except Electronics — it is currently the weakest overall performer on both dimensions.

What management should investigate further: (1) why Electronics carries such a thin margin relative to its scale — is it competitive pricing pressure, high component costs, or heavy discounting on these specific items — and whether the margin can be improved without losing the volume; (2) whether Food & Beverage's low revenue reflects genuinely low customer demand, insufficient shelf space/assortment, or pricing that is too low relative to cost, since it is underperforming on both sales and margin simultaneously; and (3) whether Home & Kitchen's strong combination of sales and margin can be replicated by promoting it further, or by applying lessons from its pricing/cost structure to other categories.

Q9. Can the Company Meet Sales Targets?

Answer: Yes, for some store-months; No, for others.
Expected finding: Some stores will exceed their targets while others will fall below them.
Why: Actual sales differ according to customer demand, transaction volume, and store performance.
Comparing Actual Sales with Targets shows whether planned performance was achieved.

Output



Interpretation

Only 9 of the 96 store-month combinations (≈9%) actually met or exceeded their target for the year — and every single one of those 9 belongs to just two stores: Kigali City Centre (8 months) and Kimironko (1 month), both urban Kigali locations. Every other store-month across the remaining six stores, for all twelve months, missed its target.

Looking at overall achievement: Kigali City Centre is the only store with an average achievement above 100% (≈109%) and the only store with a positive total variance for the year (actual sales roughly 21 million above target, cumulatively). At the other end, Muhanga Hub has both the weakest average achievement (≈42%) and the largest total negative variance (about −170 million) for the year.

Store-months that exceeded target: all 8 at Kigali City Centre, plus 1 at Kimironko (March).

Store-months that failed to reach target: all remaining 87 combinations (6 of the 8 stores missed their target in every single month of the year).

Strongest overall target achievement: Kigali City Centre.

Largest total positive variance: Kigali City Centre.

Largest total negative variance: Muhanga Hub.

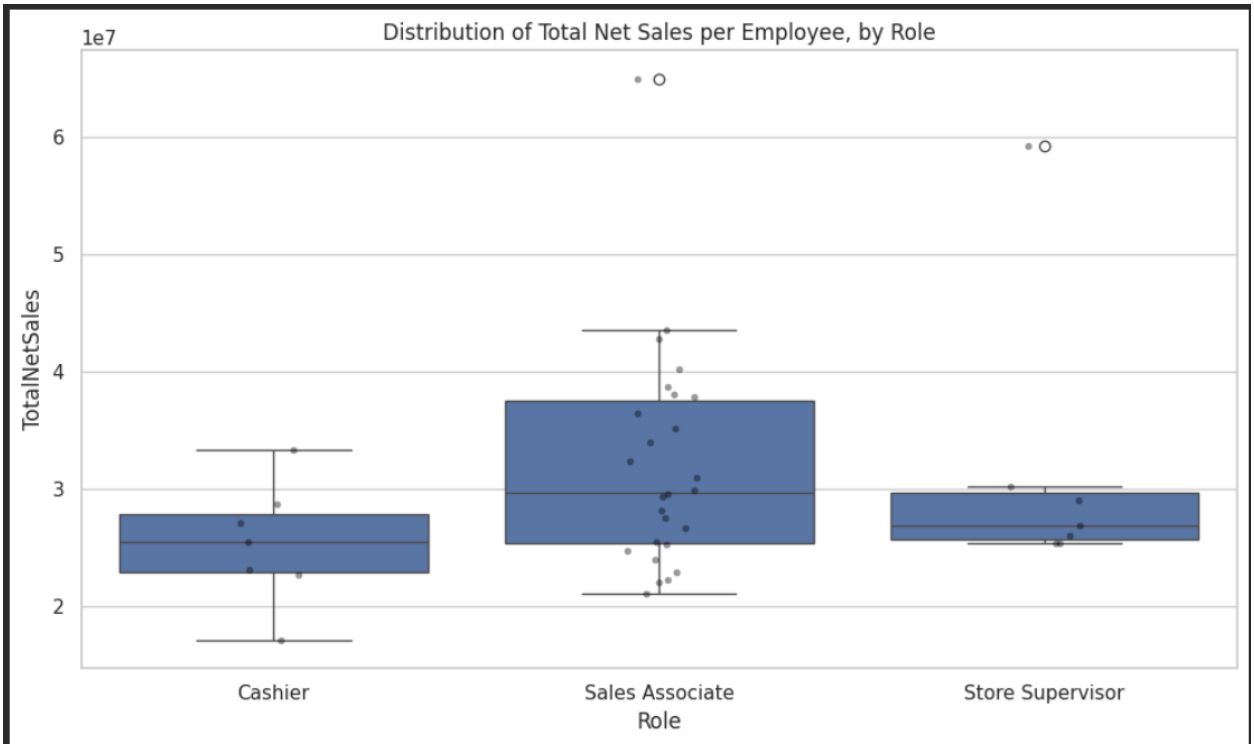
Why does target achievement tell management something raw sales figures do not? Raw Net Sales ranks stores purely by size — and by that measure, Kigali City Centre also happens to come out on top, so in this case the two views agree at the very top. But raw sales says nothing

about whether a store is meeting the expectations set for it. Target achievement reveals a much more urgent story that Net Sales alone hides completely: six of the eight stores are missing target every single month of the year, several by a wide margin (down to 30%–40% achievement in their worst months). A manager looking only at the Net Sales ranking would see "8 stores, roughly ordered from strong to weaker" and might conclude performance is broadly fine; the target-achievement view instead shows a company-wide shortfall concentrated outside the two Kigali city stores, which calls for investigating whether targets for the secondary-city stores were set unrealistically, or whether those stores genuinely need operational support.

Q10. Employee Performance and Management Decision

Answer: No, sales alone cannot determine the best employee.
Expected finding: The employee with the highest Net Sales may not have the highest Gross Profit or Average Transaction Value.
Why: Employees have different roles, transaction volumes, and store assignments. Performance should therefore be compared using sales, transactions, quantity, Gross Profit, and Profit Margin.

Output



Interpretation

There are observable differences between employees and between roles, but they line up closely with store assignment and role, not obviously with individual skill:

Store Supervisors handle the most transactions on average (≈653), followed by Sales Associates (≈631) and Cashiers (≈573) — yet Cashiers have the highest average profit margin (≈28.4%), ahead of Sales Associates (≈26.5%) and Store Supervisors (≈26.2%). Handling more transactions does not translate into the highest margin.

The employees with the highest total Net Sales, the highest average transaction value, and the highest Gross Profit are three different people in three different roles and stores — no single employee dominates on every measure, and the top performers by total sales work at the busiest store (Kigali City Centre / Store 8, Muhanga Hub), which mechanically gives them more transactions to work with.

The scatter of Transactions vs Total Net Sales shows total sales rising with the number of transactions handled, which is expected — an employee working more transactions will tend to generate more total sales almost by definition, regardless of individual skill.

Why should management be cautious about interpreting employee-level performance using sales alone?

Store assignment confounds the comparison. Employees at a busier store (more footfall, more transactions available to handle) will naturally rack up higher total Net Sales than an equally skilled employee at a quieter store — the dataset cannot separate "store traffic" from "individual selling skill."

Role changes what "high performance" should even look like. Store Supervisors may spend time on supervisory duties not captured as sales transactions, so comparing their total Net Sales directly against Sales Associates or Cashiers, who spend all their time processing transactions, is not a fair comparison of individual sales ability.

Volume and margin can move in opposite directions. The role with the most transactions per employee (Store Supervisors) has the lowest average margin among the three roles, while the role with fewer transactions (Cashiers) has the highest margin — total sales alone would credit the wrong role/employee with the "best" performance if margin is also part of what management cares about.

Management implication: before rewarding or penalising individual employees, performance should be benchmarked within the same role and, ideally, the same store, and should combine transaction volume, average transaction value, and profit margin rather than total Net Sales in isolation.

```
import pandas as pd

FILE = "/content/drive/MyDrive/Business_Intelligence/BI_Retail_Case_Study.xlsx"   # no trailing slash
xls = pd.ExcelFile(FILE)
print(xls.sheet_names)
```

```
['SalesRaw', 'DimProduct', 'DimCustomer', 'DimStore', 'DimEmployee', 'DimDate', 'Targets']
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

FILE = "/content/drive/MyDrive/Business_Intelligence/BI_Retail_Case_Study.xlsx"

# See all sheets
xls = pd.ExcelFile(FILE)
print(xls.sheet_names)

# Load each sheet (adjust names to what you see)
sales   = pd.read_excel(FILE, sheet_name="SalesRaw")
products= pd.read_excel(FILE, sheet_name="DimProduct")
stores  = pd.read_excel(FILE, sheet_name="DimStore")
targets = pd.read_excel(FILE, sheet_name="Targets")
# Employees / Customers may be in SalesRaw or a Dim table — check.
```

```
['SalesRaw', 'DimProduct', 'DimCustomer', 'DimStore', 'DimEmployee', 'DimDate', 'Targets']
```

Q1: Spearman correlation, the top products, which products are high-sales/low-profit

```
df = sales.merge(products, on="ProductID", how="left")

prod = df.groupby(["ProductID","ProductName","Category"]).agg(
    TotalNetSales = ("NetSales","sum"),
    TotalQty      = ("Quantity","sum"),
).reset_index()

# Cost = UnitCost * Quantity  (per transaction, summed)
df["TotalCost"] = df["UnitCost"] * df["Quantity"]
cost = df.groupby("ProductID")["TotalCost"].sum().reset_index()
prod = prod.merge(cost, on="ProductID")

prod["GrossProfit"] = prod["TotalNetSales"] - prod["TotalCost"]
prod["ProfitMargin"] = prod["GrossProfit"] / prod["TotalNetSales"]

# Rankings
prod["SalesRank"]  = prod["TotalNetSales"].rank(ascending=False)
prod["ProfitRank"] = prod["GrossProfit"].rank(ascending=False)
prod["RankGap"]    = prod["SalesRank"] - prod["ProfitRank"]

# Correlation between the two rankings
print(prod[["TotalNetSales","GrossProfit"]].corr(method="spearman"))

# High sales, low profit
high_sales_low_profit = prod[(prod.SalesRank <= 10) & (prod.ProfitRank > 20)]
# Low sales, high profit
low_sales_high_profit = prod[(prod.SalesRank > 20) & (prod.ProfitRank <= 10)]
```

```
               TotalNetSales  GrossProfit
TotalNetSales        1.00000      0.98833
GrossProfit          0.98833      1.00000
```

Q2: correlation values, the discount-band table

```
sales_with_cost = sales.merge(products[['ProductID', 'StandardUnitPrice']], on='ProductID', how='left')
sales_with_cost['TransactionCost'] = sales_with_cost['Quantity'] * sales_with_cost['StandardUnitPrice']
sales_with_cost['GrossProfit'] = sales_with_cost['NetSales'] - sales_with_cost['TransactionCost']

q2 = sales_with_cost[["DiscountPct","DiscountAmount","GrossSales","NetSales",
            "Quantity","GrossProfit"]].copy()

print(q2.describe())
```

```
print(q2.corr(numeric_only=True))

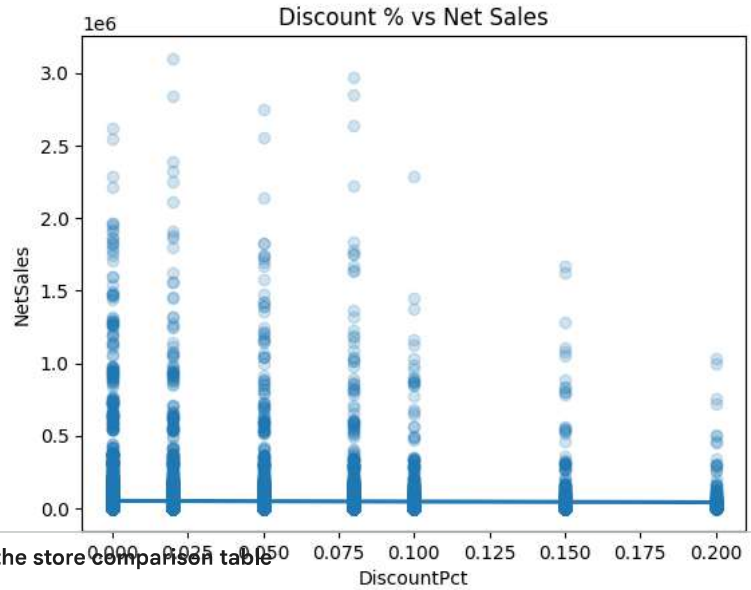
# Visual 1: discount vs net sales (scatter + trend)
sns.regplot(data=q2, x="DiscountPct", y="NetSales", scatter_kws={"alpha":0.2})
plt.title("Discount % vs Net Sales"); plt.show()

# Visual 2: bucket transactions by discount level
q2["DiscBand"] = pd.cut(q2.DiscountPct,
                        bins=[-0.001,0,0.05,0.10,0.15,0.20,1],
                        labels=["0%","0-5%","5-10%","10-15%","15-20%",">20%"])
band = q2.groupby("DiscBand").agg(
    AvgNetSales=("NetSales","mean"),
    AvgQty=("Quantity","mean"),
    AvgGrossProfit=("GrossProfit","mean"),
    N=("NetSales","size"))
print(band)
band.plot(kind="bar", subplots=True, figsize=(10,8)); plt.tight_layout(); plt.show()
```

```
        DiscountPct  DiscountAmount    GrossSales      NetSales      Quantity  \
count  24975.000000    25035.000000  2.503500e+04  2.503500e+04  25035.000000
mean       0.041958     2170.118272  5.147581e+04  4.930569e+04      2.592610
std        0.046212     9513.237568  1.523395e+05  1.461956e+05      1.897384
min        0.000000        0.000000  8.010000e+02  6.496000e+02      1.000000
25%        0.000000        0.000000  7.673500e+03  7.331740e+03      1.000000
50%        0.020000      342.480000  1.768100e+04  1.701000e+04      2.000000
75%        0.080000     1390.500000  4.048500e+04  3.858900e+04      3.000000
max        0.200000   295181.100000  3.230810e+06  3.099113e+06     10.000000

         GrossProfit
count   25035.000000
mean    -1627.396722
std     13888.364029
min   -360538.200000
25%     -1636.040000
50%      -386.540000
75%       103.150000
max    362670.000000
                DiscountPct  DiscountAmount  GrossSales  NetSales  Quantity  \
DiscountPct        1.000000        0.251188    0.000922 -0.015371  0.000553
DiscountAmount     0.251188        1.000000    0.664020  0.626853  0.166737
GrossSales         0.000922        0.664020    1.000000  0.998816  0.246910
NetSales          -0.015371        0.626853    0.998816  1.000000  0.246437
Quantity           0.000553        0.166737    0.246910  0.246437  1.000000
GrossProfit       -0.172160       -0.603369   -0.333275 -0.308019 -0.097622

                GrossProfit
DiscountPct       -0.172160
DiscountAmount    -0.603369
GrossSales        -0.333275
NetSales          -0.308019
Quantity          -0.097622
GrossProfit        1.000000
```



Q3: the store comparison table

```
st = sales_with_cost.merge(stores, on="StoreID", how="left")

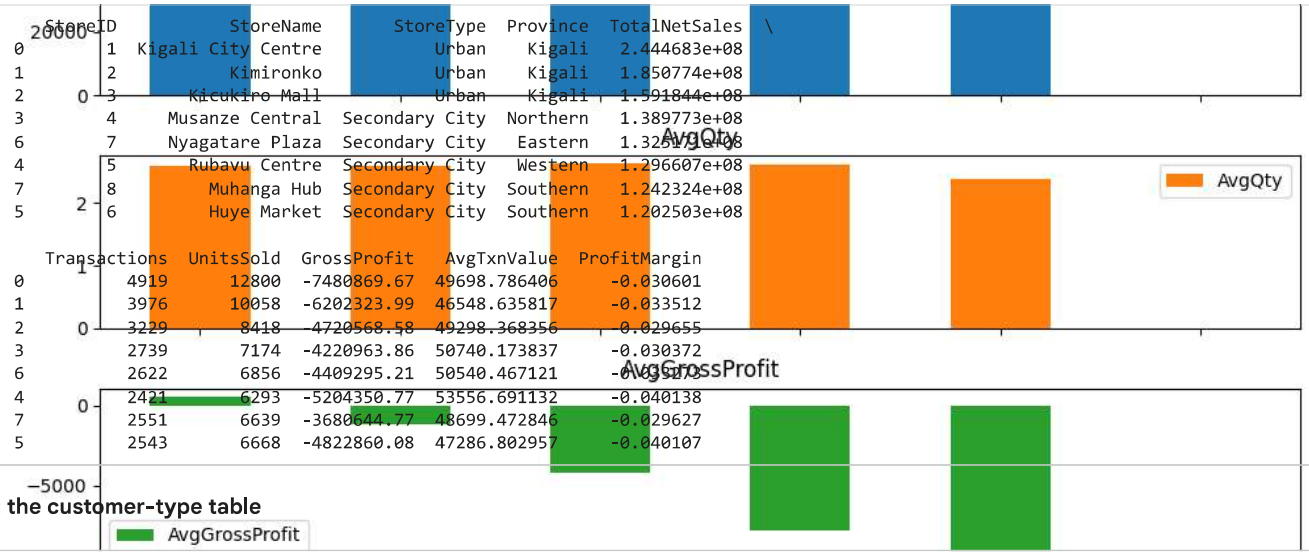
store = st.groupby(["StoreID","StoreName","StoreType","Province"]).agg(
```

```
    TotalNetSales = ("NetSales","sum"),
    Transactions   = ("TransactionID","nunique"),
    UnitsSold     = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
).reset_index()

store["AvgTxnValue"]  = store.TotalNetSales / store.Transactions
store["ProfitMargin"] = store.GrossProfit / store.TotalNetSales
print(store.sort_values("TotalNetSales", ascending=False))
```

```
  StoreID          StoreName      StoreType  Province  TotalNetSales  \
0       1  Kigali City Centre          Urban    Kigali   2.444683e+08
1       2           Kimironko          Urban    Kigali   1.850774e+08
2       3       Kicukiro Mall          Urban    Kigali   1.591844e+08
3       4     Musanze Central  Secondary City  Northern   1.389773e+08
6       7     Nyagatare Plaza  Secondary City   Eastern   1.325173e+08
4       5       Rubavu Centre  Secondary City   Western   1.296607e+08
7       8        Muhanga Hub  Secondary City  Southern   1.242324e+08
5       6        Huye Market  Secondary City  Southern   1.202503e+08

   Transactions  UnitsSold  GrossProfit   AvgTxnValue  ProfitMargin
0          4919      12800  -7480869.67  49698.786406     -0.030601
1          3976      10058  -6202323.99  46548.635817     -0.033512
2          3229       8418  -4720568.58  49298.368356     -0.029655
3          2739       7174  -4220963.86  50740.173837     -0.030372
6          2622       6856  -4409295.21  50540.467121     -0.033273
4          2421       6293  -5204350.77  53556.691132     -0.040138
7          2551       6639  -3680644.77  48699.472846     -0.029627
5          2543       6668  -4822860.08  47286.802957     -0.040107
```



Q4: the customer-type table

```
customers = pd.read_excel(FILE, sheet_name="DimCustomer")

sales_customer_data = sales_with_cost.merge(customers[['CustomerID', 'CustomerType']], on='CustomerID', how='left')

cust = sales_customer_data.groupby("CustomerType").agg(
    Customers     = ("CustomerID","nunique"),
    Transactions  = ("TransactionID","nunique"),
    TotalNetSales = ("NetSales","sum"),
    TotalQty      = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
).reset_index()

cust["PctNetSales"]   = cust.TotalNetSales / cust.TotalNetSales.sum()*100
cust["AvgTxnValue"]   = cust.TotalNetSales / cust.Transactions
cust["ProfitMargin"]  = cust.GrossProfit / cust.TotalNetSales
print(cust)
```

```
  CustomerType  Customers  Transactions  TotalNetSales  TotalQty  GrossProfit  \
0   Individual       3600         18146   8.926403e+08     47176 -30331535.14
1  Institution        256          1271   7.229076e+07      3350  -2352637.67
2          SME       1119          5583   2.694370e+08     14380  -8057704.12

   PctNetSales   AvgTxnValue  ProfitMargin
0    72.315569  49192.123050     -0.033980
1     5.856500  56877.075004     -0.032544
2    21.827931  48260.253606     -0.029906
```

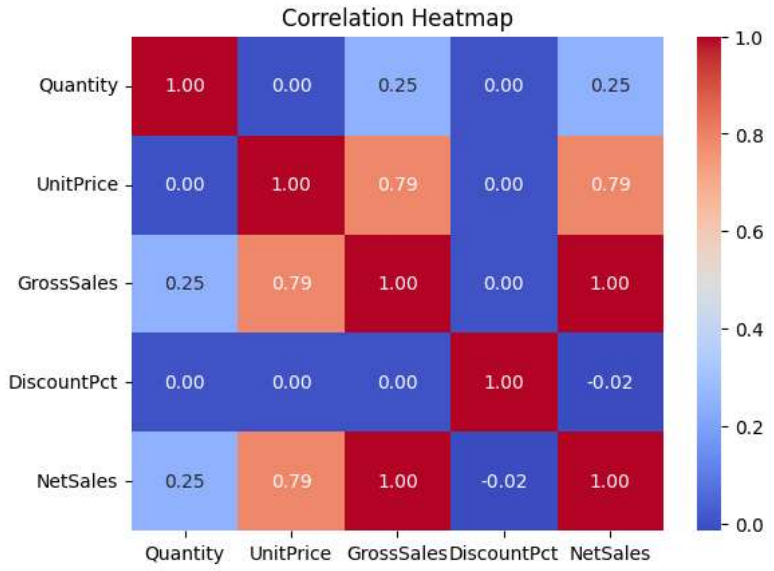
Q5: the correlation matrix

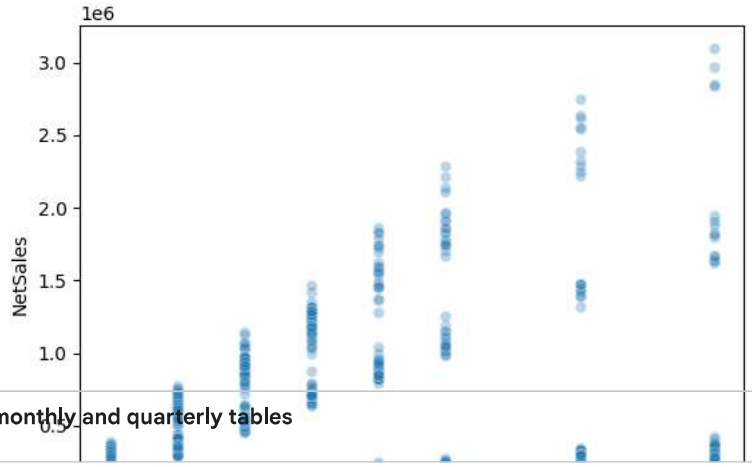
```
q5 = sales[["Quantity","UnitPrice","GrossSales","DiscountPct","NetSales"]].copy()
print(q5.describe())
corr = q5.corr(numeric_only=True)
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap"); plt.show()

# Visual 1: Quantity vs NetSales
sns.scatterplot(data=q5, x="Quantity", y="NetSales", alpha=0.3); plt.show()
# Visual 2: UnitPrice vs NetSales (colored by discount)
sns.scatterplot(data=q5, x="UnitPrice", y="NetSales", hue="DiscountPct", alpha=0.4); plt.show()
```

```
           Quantity      UnitPrice    GrossSales   DiscountPct      NetSales
count  25035.000000   24985.000000  2.503500e+04  24975.000000  2.503500e+04
mean       2.592610   19806.217210  5.147581e+04      0.041958  4.930569e+04
std        1.897384   45961.762042  1.523395e+05      0.046212  1.461956e+05
min        1.000000     791.000000  8.010000e+02      0.000000  6.496000e+02
25%        1.000000    4204.000000  7.673500e+03      0.000000  7.331740e+03
50%        2.000000    8330.000000  1.768100e+04      0.020000  1.701000e+04
75%        3.000000   18089.000000  4.048500e+04      0.080000  3.858900e+04
max       10.000000  387691.000000  3.230810e+06      0.200000  3.099113e+06
             Quantity  UnitPrice  GrossSales  DiscountPct  NetSales
Quantity     1.000000   0.000259    0.246910     0.000553  0.246437
UnitPrice    0.000259   1.000000    0.789778     0.002740  0.788208
GrossSales   0.246910   0.789778    1.000000     0.000922  0.998816
DiscountPct  0.000553   0.002740    0.000922     1.000000 -0.015371
NetSales     0.246437   0.788208    0.998816    -0.015371  1.000000
```





Q6: monthly and quarterly tables

```
# ============================================================
# QUESTION 6 — Does the Company Have a Monthly Sales Pattern?
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt

# ---------- 1. Load data ----------
sales   = pd.read_excel(FILE, sheet_name="SalesRaw")
dimdate = pd.read_excel(FILE, sheet_name="DimDate")

# Ensure datetime types
sales["TransactionDate"] = pd.to_datetime(sales["TransactionDate"])
dimdate["Date"]          = pd.to_datetime(dimdate["Date"])   # adjust name if needed

# ---------- 2. Merge sales with date dimension ----------
q6 = sales.merge(dimdate, left_on="TransactionDate", right_on="Date", how="left")

# ---------- 3. Monthly aggregates ----------
monthly = q6.groupby(["Year","MonthNumber","Month","YearMonth"]).agg(
    MonthlyNetSales=("NetSales","sum"),
```

```
    TransactionCount=("TransactionID","nunique"),
    QuantitySold=("Quantity","sum"),
).reset_index().sort_values(["Year","MonthNumber"])

monthly["AvgTransactionValue"] = monthly.MonthlyNetSales / monthly.TransactionCount
monthly["MoM_PctChange"]       = monthly.MonthlyNetSales.pct_change() * 100

print("===== MONTHLY AGGREGATES =====")
print(monthly)

# ---------- 4. Quarterly aggregates ----------
quarterly = q6.groupby(["Year","Quarter"]).agg(
    QuarterlyNetSales=("NetSales","sum"),
    Q_Transactions=("TransactionID","nunique"),
    Q_Quantity=("Quantity","sum"),
).reset_index().sort_values(["Year","Quarter"])

quarterly["QoQ_PctChange"] = quarterly.QuarterlyNetSales.pct_change() * 100

print("\n===== QUARTERLY AGGREGATES =====")
print(quarterly)

# ---------- 5. Headline answers ----------
print("\n===== HEADLINE ANSWERS =====")
print("Highest month:", monthly.loc[monthly.MonthlyNetSales.idxmax(),
      ["YearMonth","MonthlyNetSales"]].to_dict())
print("Lowest month :", monthly.loc[monthly.MonthlyNetSales.idxmin(),
      ["YearMonth","MonthlyNetSales"]].to_dict())
print("Best quarter :", quarterly.loc[quarterly.QuarterlyNetSales.idxmax(),
      ["Year","Quarter","QuarterlyNetSales"]].to_dict())
print("Largest + MoM:", monthly.loc[monthly.MoM_PctChange.idxmax(),
      ["YearMonth","MoM_PctChange"]].to_dict())
print("Largest - MoM:", monthly.loc[monthly.MoM_PctChange.idxmin(),
      ["YearMonth","MoM_PctChange"]].to_dict())

# ---------- 6. Visualization 1: Monthly Net Sales & Transaction Volume ----------
fig, ax1 = plt.subplots(figsize=(14,6))
ax1.plot(monthly.YearMonth, monthly.MonthlyNetSales, marker="o", color="steelblue")
ax1.set_xlabel("Month"); ax1.set_ylabel("Net Sales", color="steelblue")
ax1.tick_params(axis="x", rotation=45)

ax2 = ax1.twinx()
ax2.bar(monthly.YearMonth, monthly.TransactionCount, alpha=0.25, color="gray")
ax2.set_ylabel("Transaction Count", color="gray")

plt.title("Monthly Net Sales & Transaction Volume — 2025")
fig.tight_layout(); plt.show()

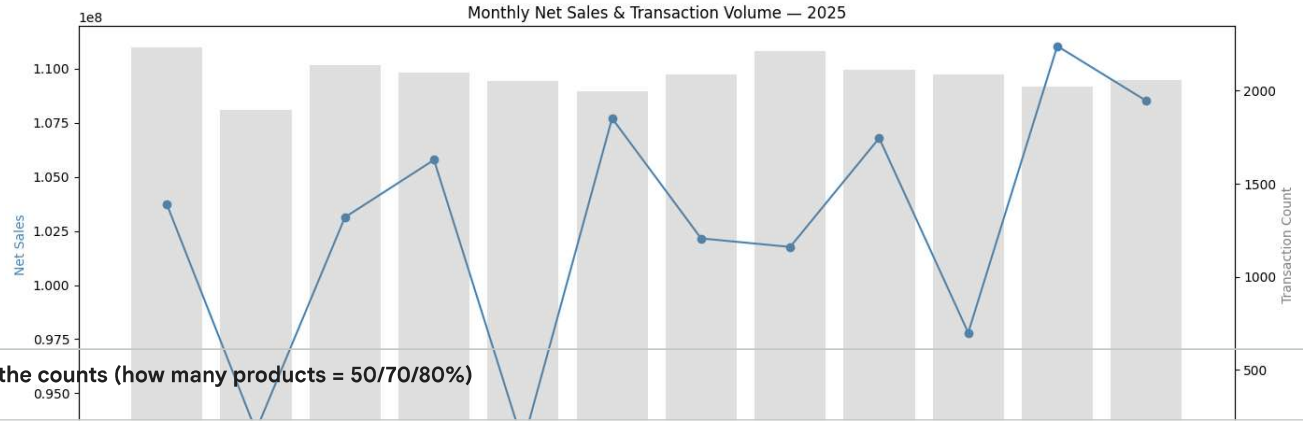
# ---------- 7. Visualization 2: Month-over-Month % Change ----------
plt.figure(figsize=(14,5))
plt.bar(monthly.YearMonth, monthly.MoM_PctChange, color="indianred")
plt.axhline(0, color="black", linewidth=0.8)
plt.xticks(rotation=45); plt.ylabel("MoM % Change")
plt.title("Month-over-Month % Change in Net Sales"); plt.show()
```

```
===== MONTHLY AGGREGATES =====
    Year  MonthNumber      Month YearMonth  MonthlyNetSales  TransactionCount  \
0   2025            1    January   2025-01     1.037253e+08              2235
1   2025            2   February   2025-02     9.327233e+07              1896
2   2025            3      March   2025-03     1.031316e+08              2138
3   2025            4      April   2025-04     1.057824e+08              2097
4   2025            5        May   2025-05     9.269844e+07              2052
5   2025            6       June   2025-06     1.077024e+08              1997
6   2025            7       July   2025-07     1.021554e+08              2086
7   2025            8     August   2025-08     1.017619e+08              2214
8   2025            9  September   2025-09     1.067885e+08              2114
9   2025           10    October   2025-10     9.780096e+07              2089
10  2025           11   November   2025-11     1.110400e+08              2023
11  2025           12   December   2025-12     1.085089e+08              2059

    QuantitySold  AvgTransactionValue  MoM_PctChange
0           5878         46409.551772            NaN
1           4861         49194.268465     -10.077590
2           5583         48237.405061      10.570379
3           5281         50444.628679       2.570323
4           5336         45174.679922     -12.368735
5           5262         53932.118438      16.185814
6           5379         48971.911779      -5.150331
7           5880         45962.899246      -0.385245
8           5504         50514.884456       4.939578
9           5415         46817.117391      -8.416178
10          5177         54888.756233      13.536673
11          5350         52699.783895      -2.279449

===== QUARTERLY AGGREGATES =====
   Year Quarter  QuarterlyNetSales  Q_Transactions  Q_Quantity  QoQ_PctChange
0  2025      Q1       3.001293e+08            6269       16322            NaN
1  2025      Q2       3.061833e+08            6146       15879       2.017137
2  2025      Q3       3.107057e+08            6414       16763       1.477044
3  2025      Q4       3.173498e+08            6171       15942       2.138369

===== HEADLINE ANSWERS =====
Highest month: {'YearMonth': '2025-11', 'MonthlyNetSales': 111039953.86}
Lowest month : {'YearMonth': '2025-05', 'MonthlyNetSales': 92698443.2}
Best quarter : {'Year': 2025, 'Quarter': 'Q4', 'QuarterlyNetSales': 317349767.13}
Largest + MoM: {'YearMonth': '2025-06', 'MoM_PctChange': 16.18581370091445}
Largest - MoM: {'YearMonth': '2025-05', 'MoM_PctChange': -12.368735091630755}
```



Q7: the counts (how many products = 50/70/80%)

```
sales_product_data = sales_with_cost.merge(products[['ProductID', 'ProductName', 'Category']], on='ProductID', how='left')

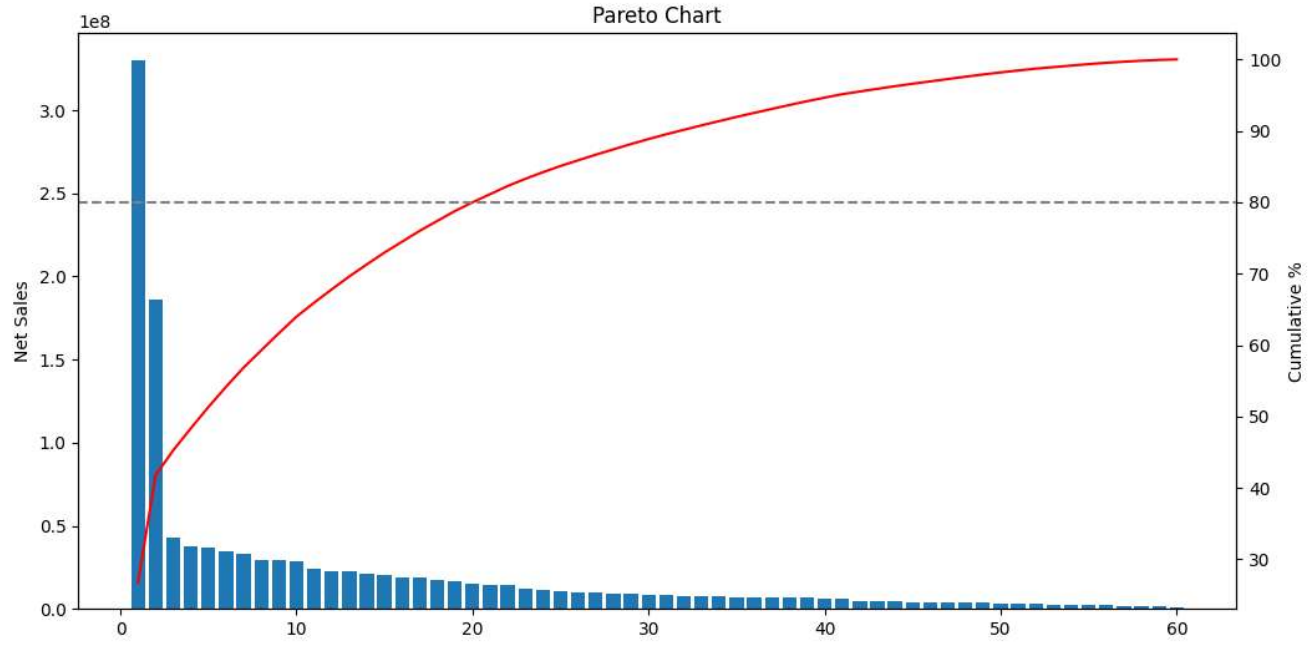
p = sales_product_data.groupby(["ProductID","ProductName","Category"])["NetSales"].sum() \
        .reset_index().sort_values("NetSales", ascending=False)
p["Pct"]       = p.NetSales / p.NetSales.sum()*100
p["CumPct"]    = p.Pct.cumsum()
p["Rank"]      = range(1, len(p)+1)

n50 = (p.CumPct <= 50).sum() + 1
n70 = (p.CumPct <= 70).sum() + 1
n80 = (p.CumPct <= 80).sum() + 1
print(f"Products for 50%: {n50}, 70%: {n70}, 80%: {n80} of {len(p)}")

fig, ax1 = plt.subplots(figsize=(12,6))
ax1.bar(p.Rank, p.NetSales); ax1.set_ylabel("Net Sales")
ax2 = ax1.twinx()
ax2.plot(p.Rank, p.CumPct, color="red"); ax2.axhline(80, ls="--", color="gray")
ax2.set_ylabel("Cumulative %"); plt.title("Pareto Chart"); plt.show()
```



```
Products for 50%: 5, 70%: 14, 80%: 21 of 60
```



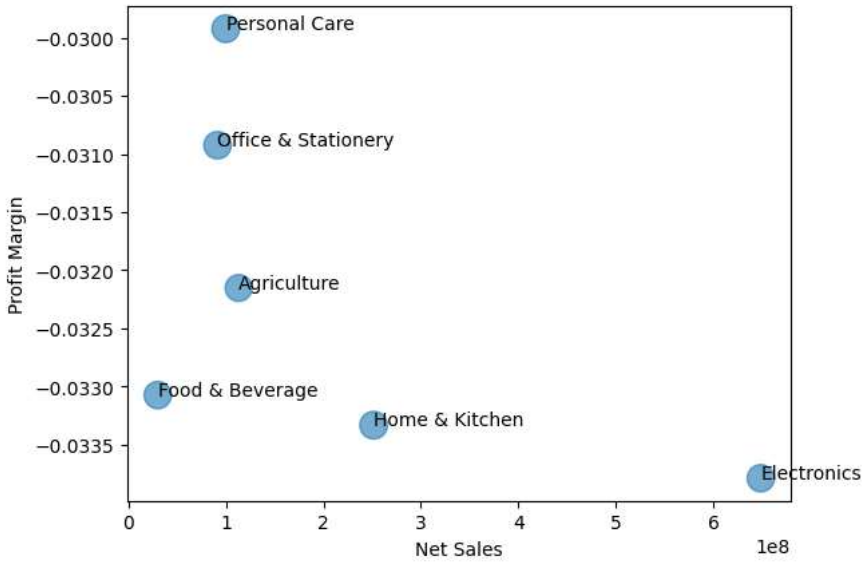
Q8: the category table

```
cat = sales_product_data.groupby("Category").agg(
    TotalNetSales = ("NetSales","sum"),
    Quantity       = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
    NumProducts   = ("ProductID","nunique"),
).reset_index()
cat["PctNetSales"]  = cat.TotalNetSales / cat.TotalNetSales.sum()*100
cat["ProfitMargin"] = cat.GrossProfit / cat.TotalNetSales
print(cat)

# Bubble chart: sales vs margin, size = quantity
plt.scatter(cat.TotalNetSales, cat.ProfitMargin, s=cat.Quantity/50, alpha=0.6)
for _,r in cat.iterrows(): plt.annotate(r.Category,(r.TotalNetSales,r.ProfitMargin))
plt.xlabel("Net Sales"); plt.ylabel("Profit Margin"); plt.show()
```

```
              Category  TotalNetSales  Quantity  GrossProfit  NumProducts  \
0          Agriculture   1.129778e+08     10401  -3632683.50           10
1          Electronics   6.488274e+08     10762 -21924631.50           10
2      Food & Beverage   2.997370e+07     10763   -991402.67           10
3       Home & Kitchen   2.515526e+08     11075  -8385396.97           10
4  Office & Stationery   9.129694e+07     10858  -2823360.20           10
5        Personal Care   9.973960e+07     11047  -2984402.09           10

   PctNetSales  ProfitMargin
0     9.152685     -0.032154
1    52.563527     -0.033791
2     2.428263     -0.033076
3    20.379060     -0.033335
4     7.396250     -0.030925
5     8.080216     -0.029922
```



Q9: the store target-achievement table

```
sales["Month"] = pd.to_datetime(sales.TransactionDate).dt.to_period("M").astype(str)

# Convert 'Month' in targets to the same format as in sales before merging
targets['Month'] = pd.to_datetime(targets['Month']).dt.to_period("M").astype(str)

actual = sales.groupby(["Month","StoreID"])["NetSales"].sum().reset_index()
actual = actual.merge(targets, on=["Month","StoreID"], how="left")
actual = actual.merge(stores[["StoreID","StoreName"]], on="StoreID", how="left")

actual["Variance"]    = actual.NetSales - actual.SalesTarget
actual["Achievement"] = actual.NetSales / actual.SalesTarget * 100
print(actual)

# Who exceeded / missed
print(actual[actual.Variance > 0])
print(actual[actual.Variance < 0])

# By store
by_store = actual.groupby("StoreName").agg(
    TotalActual=("NetSales","sum"),
    TotalTarget=("SalesTarget","sum"),
    TotalVariance=("Variance","sum")).reset_index()
by_store["OverallAchievement"] = by_store.TotalActual / by_store.TotalTarget * 100
print(by_store.sort_values("OverallAchievement", ascending=False))
# Largest positive / negative variance
print(by_store.sort_values("TotalVariance", ascending=False).head(1))
print(by_store.sort_values("TotalVariance").head(1))

# Visualization
pivot = actual.pivot_table(index="Month", columns="StoreName",
                           values=["NetSales","SalesTarget"])
pivot.plot(kind="bar", figsize=(14,7)); plt.title("Actual vs Target by Store-Month")
plt.tight_layout(); plt.show()
```

```
      Month  StoreID     NetSales  SalesTarget          StoreName  \
0   2025-01        1  23563586.51     17464775  Kigali City Centre
1   2025-01        2  13072065.77     18074483          Kimironko
2   2025-01        3  14430248.64     23763965       Kicukiro Mall
3   2025-01        4  11546387.91     22645508     Musanze Central
4   2025-01        5   8305590.91     19988105       Rubavu Centre
..      ...      ...          ...          ...                 ...
91  2025-12        4  14453129.24     25634715     Musanze Central
92  2025-12        5  10984084.11     22626535       Rubavu Centre
93  2025-12        6   9582844.30     14897761        Huye Market
94  2025-12        7  10251493.41     23657590     Nyagatare Plaza
95  2025-12        8  13247729.14     26039268         Muhanga Hub

       Variance  Achievement
0    6098811.51   134.920642
1   -5002417.23    72.323318
2   -9333716.36    60.723236
3  -11099120.09    50.987542
4  -11682514.09    41.552668
..          ...          ...
91 -11181585.76    56.381080
92 -11642450.89    48.545144
93  -5314916.70    64.324057
94 -13406096.59    43.332788
95 -12791538.86    50.875966

[96 rows x 7 columns]
      Month  StoreID     NetSales  SalesTarget          StoreName  \
0   2025-01        1  23563586.51     17464775  Kigali City Centre
16  2025-03        1  18906529.42     17883929  Kigali City Centre
17  2025-03        2  18715576.53     18508271          Kimironko
40  2025-06        1  22893771.81     18512661  Kigali City Centre
48  2025-07        1  22945313.99     18722239  Kigali City Centre
56  2025-08        1  21639216.65     18931816  Kigali City Centre
64  2025-09        1  24156153.77     19141393  Kigali City Centre
80  2025-11        1  23476911.75     19560548  Kigali City Centre
88  2025-12        1  25191541.40     19770125  Kigali City Centre

      Variance  Achievement
0   6098811.51   134.920642
16  1022600.42   105.717985
17   207305.53   101.120070
40  4381110.81   123.665484
48  4223074.99   122.556463
56  2707400.65   114.300797
64  5014760.77   126.198515
80  3916363.75   120.021749
88  5421416.40   127.422267
      Month  StoreID     NetSales  SalesTarget        StoreName     Variance  \
1   2025-01        2  13072065.77     18074483        Kimironko  -5002417.23
2   2025-01        3  14430248.64     23763965    Kicukiro Mall  -9333716.36
3   2025-01        4  11546387.91     22645508  Musanze Central -11099120.09
4   2025-01        5   8305590.91     19988105    Rubavu Centre -11682514.09
5   2025-01        6   9793196.25     13160566      Huye Market  -3367369.75
..      ...      ...          ...          ...              ...          ...
91  2025-12        4  14453129.24     25634715  Musanze Central -11181585.76
92  2025-12        5  10984084.11     22626535    Rubavu Centre -11642450.89
93  2025-12        6   9582844.30     14897761      Huye Market  -5314916.70
94  2025-12        7  10251493.41     23657590  Nyagatare Plaza -13406096.59
95  2025-12        8  13247729.14     26039268      Muhanga Hub -12791538.86
```

Q10: the employee table and role averages

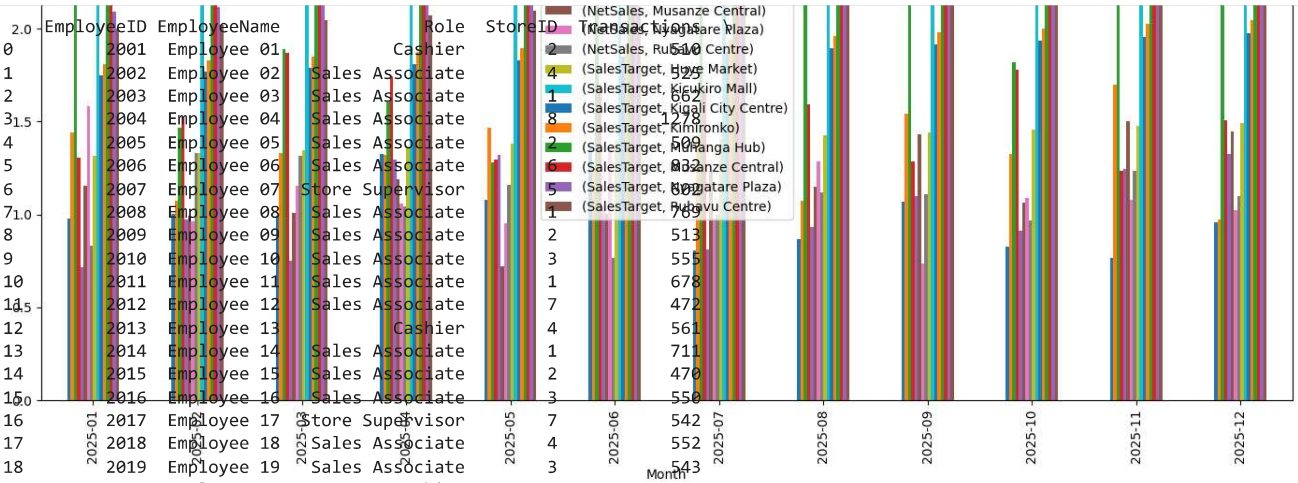
```
employees = pd.read_excel(FILE, sheet_name="DimEmployee")
sales_employee_data = sales_with_cost.merge(employees[['EmployeeID', 'EmployeeName', 'Role']], on='EmployeeID', how='left')

emp = sales_employee_data.groupby(["EmployeeID","EmployeeName","Role","StoreID"]).agg(
    Transactions  = ("TransactionID","nunique"),
    TotalNetSales = ("NetSales","sum"),
    TotalQty      = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
).reset_index()

emp["AvgTxnValue"]  = emp.TotalNetSales / emp.Transactions
emp["ProfitMargin"] = emp.GrossProfit / emp.TotalNetSales
print(emp)

# Compare within roles
print(emp.groupby("Role")[["Transactions","TotalNetSales","AvgTxnValue","GrossProfit","ProfitMargin"]].mean())

# Top performers
print(emp.loc[emp.TotalNetSales.idxmax()])
print(emp.loc[emp.AvgTxnValue.idxmax()])
print(emp.loc[emp.GrossProfit.idxmax()])
```




```
    EmployeeID EmployeeName            Role  StoreID  Transactions  \
0         2001  Employee 01         Cashier        2           540
1         2002  Employee 02   Sales Associate        4           525
2         2003  Employee 03   Sales Associate        1           662
3         2004  Employee 04   Sales Associate        8          1278
4         2005  Employee 05   Sales Associate        2           509
5         2006  Employee 06   Sales Associate        6           832
6         2007  Employee 07  Store Supervisor        5           622
7         2008  Employee 08   Sales Associate        1           769
8         2009  Employee 09   Sales Associate        2           513
9         2010  Employee 10   Sales Associate        3           555
10        2011  Employee 11   Sales Associate        1           678
11        2012  Employee 12   Sales Associate        7           472
12        2013  Employee 13         Cashier        4           561
13        2014  Employee 14   Sales Associate        1           711
14        2015  Employee 15   Sales Associate        2           470
15        2016  Employee 16   Sales Associate        3           550
16        2017  Employee 17  Store Supervisor        7           542
17        2018  Employee 18   Sales Associate        4           552
18        2019  Employee 19   Sales Associate        3           543
19        2020  Employee 20         Cashier        1           688
20        2021  Employee 21  Store Supervisor        4           544
21        2022  Employee 22   Sales Associate        1           729
22        2023  Employee 23   Sales Associate        4           557
23        2024  Employee 24  Store Supervisor        3           516
24        2025  Employee 25   Sales Associate        6           848
25        2026  Employee 26  Store Supervisor        8          1273
26        2027  Employee 27  Store Supervisor        5           578
27        2028  Employee 28   Sales Associate        6           863
28        2029  Employee 29   Sales Associate        7           530
29        2030  Employee 30   Sales Associate        7           526
30        2031  Employee 31         Cashier        2           479
31        2032  Employee 32   Sales Associate        5           628
32        2033  Employee 33   Sales Associate        5           613
33        2034  Employee 34  Store Supervisor        2           518
34        2035  Employee 35   Sales Associate        2           469
35        2036  Employee 36   Sales Associate        2           508
36        2037  Employee 37         Cashier        3           540
37        2038  Employee 38   Sales Associate        3           525
38        2039  Employee 39         Cashier        1           682
39        2040  Employee 40         Cashier        7           552

    TotalNetSales  TotalQty  GrossProfit   AvgTxnValue  ProfitMargin
0     17090349.98      1243   -530150.02  33510.490157     -0.031020
1     29507510.42      1336  -1148689.58  56204.781752     -0.038929
2     35209959.12      1746  -1454840.88  53187.249426     -0.041319
3     64955495.93      3323  -1347304.07  50825.896659     -0.020742
4     22926957.30      1224  -1067142.70  45043.138114     -0.046545
5     37871278.80      2165  -1502321.20  45518.363942     -0.039669
6     29055090.09      1610  -1571609.91  48264.269252     -0.054091
7     40187279.00      2030  -1145621.00  52259.140442     -0.028507
8     25441749.80      1314   -898450.20  49594.054191     -0.035314
9     28162577.54      1404  -1068022.46  50743.382955     -0.037923
10    29351119.21      1693  -1228780.79  43290.736298     -0.041865
11    21043773.04      1282   -849226.96  44584.264915     -0.040355
12    22651281.43      1485   -614318.57  40376.615740     -0.027121
13    36461738.95      1890  -1429161.05  51282.333263     -0.039196
14    22054788.10      1240   -733111.90  46925.081064     -0.033240
```

```
import zipfile, glob
from google.colab import files

with zipfile.ZipFile("charts.zip", "w") as z:
    for img in glob.glob("*.png"):
        z.write(img)

files.download("charts.zip")
```

```
import zipfile, glob
from google.colab import files

with zipfile.ZipFile("assignment_outputs.zip", "w") as z:
    # tables
    if "all_outputs.xlsx" in glob.glob("*.xlsx"):
        z.write("all_outputs.xlsx")
    # text outputs
    for txt in glob.glob("*.txt"):
        z.write(txt)
    # charts
    for png in glob.glob("*.png"):
        z.write(png)
    # raw CSVs if you saved any
    for csv in glob.glob("*.csv"):
        z.write(csv)

files.download("assignment_outputs.zip")
```

```
# ============================================================
# KPI DASHBOARD — Numeric Money Columns
# NetSales, GrossProfit, SalesTarget
# ============================================================

import pandas as pd

# Define the file path
FILE = "/content/drive/MyDrive/Business_Intelligence/BI_Retail_Case_Study.xlsx"

# Load the required dataframes
sales   = pd.read_excel(FILE, sheet_name="SalesRaw")
products= pd.read_excel(FILE, sheet_name="DimProduct")
stores  = pd.read_excel(FILE, sheet_name="DimStore")
targets = pd.read_excel(FILE, sheet_name="Targets")

# --- Prep ---
sales["TransactionDate"] = pd.to_datetime(sales["TransactionDate"])
sales["Month"] = sales["TransactionDate"].dt.to_period("M").astype(str)

# Cost is required to derive GrossProfit if not already present
df = sales.merge(products[["ProductID", "UnitCost"]], on="ProductID", how="left")
df["GrossProfit"] = df["NetSales"] - (df["UnitCost"] * df["Quantity"])

# --- Aggregations ---
total_sales      = df["NetSales"].sum()
total_profit     = df["GrossProfit"].sum()
total_txn        = df["TransactionID"].nunique()
total_units      = df["Quantity"].sum()

monthly_sales    = df.groupby("Month")["NetSales"].sum()
monthly_mom      = monthly_sales.pct_change() * 100

top5_sales       = df.groupby("ProductID")["NetSales"].sum().nlargest(5).sum()
top5_profit      = df.groupby("ProductID")["GrossProfit"].sum().nlargest(5).sum()

store_sales      = df.merge(stores[["StoreID", "StoreName"]], on="StoreID", how="left") \
                     .groupby("StoreName")["NetSales"].sum()

# Targets — aggregate to same level as actual
target_total     = targets["SalesTarget"].sum()
actual_by_store  = df.groupby("StoreID")["NetSales"].sum()
target_by_store  = targets.groupby("StoreID")["SalesTarget"].sum()
ach_by_store     = (actual_by_store / target_by_store * 100)
```

```
# --- KPI dictionary ---
kpi = {
    # --- NetSales KPIs ---
    "Total Net Sales (RWF)":          total_sales,
    "Avg Transaction Value (ATV)":    total_sales / total_txn,
    "Monthly Avg Net Sales (RWF)":    monthly_sales.mean(),
    "Best Month":                     monthly_sales.idxmax(),
    "Best Month Net Sales (RWF)":     monthly_sales.max(),
    "Worst Month":                    monthly_sales.idxmin(),
    "Worst Month Net Sales (RWF)":    monthly_sales.min(),
    "Latest MoM Growth %":            monthly_mom.iloc[-1],
    "Top-5 Product Revenue Share %":  top5_sales / total_sales * 100,
    "Top Store Revenue Share %":      store_sales.max() / total_sales * 100,

    # --- GrossProfit KPIs ---
    "Total Gross Profit (RWF)":       total_profit,
    "Gross Profit Margin %":          total_profit / total_sales * 100,
    "Profit per Transaction (RWF)":   total_profit / total_txn,
    "Profit per Unit (RWF)":          total_profit / total_units,
    "Top-5 Product Profit Share %":   top5_profit / total_profit * 100 if total_profit != 0 else float("nan"),

    # --- SalesTarget KPIs ---
    "Total Sales Target (RWF)":       target_total,
    "Achievement % (Overall)":        total_sales / target_total * 100,
    "Variance vs Target (RWF)":       total_sales - target_total,
    "Variance vs Target %":           (total_sales - target_total) / target_total * 100,
    "% Stores Achieving Target":      (ach_by_store >= 100).mean() * 100,
}

kpi_df = pd.DataFrame(list(kpi.items()), columns=["KPI", "Value"])
print("=" * 60)
print("KPI DASHBOARD — MONEY COLUMNS")
print("=" * 60)

# Format nicely for readability
for _, row in kpi_df.iterrows():
    name = row["KPI"]
    val  = row["Value"]
    if isinstance(val, str):
        print(f"{name:<35} {val}")
    elif "Share" in name or "Margin" in name or "%" in name or "Growth" in name:
        print(f"{name:<35} {val:>15,.2f} %")
    else:
        print(f"{name:<35} {val:>15,.0f} RWF")
```

```
============================================================
KPI DASHBOARD — MONEY COLUMNS
============================================================
Total Net Sales (RWF)                 1,234,368,023 RWF
Avg Transaction Value (ATV)                  49,375 RWF
Monthly Avg Net Sales (RWF)             102,864,002 RWF
Best Month                              2025-11
Best Month Net Sales (RWF)              111,039,954 RWF
Worst Month                             2025-05
Worst Month Net Sales (RWF)              92,698,443 RWF
Latest MoM Growth %                           -2.28 %
Top-5 Product Revenue Share %                 51.32 %
Top Store Revenue Share %                     19.81 %
Total Gross Profit (RWF)                329,491,323 RWF
Gross Profit Margin %                         26.69 %
Profit per Transaction (RWF)                 13,180 RWF
Profit per Unit (RWF)                         5,076 RWF
Top-5 Product Profit Share %                  39.38 %
Total Sales Target (RWF)              2,033,918,030 RWF
Achievement % (Overall)                       60.69 %
Variance vs Target (RWF)              -799,550,007 RWF
Variance vs Target %                          -39.31 %
% Stores Achieving Target                     12.50 %
```

```
# ============================================================
# KPI DASHBOARD — Profit Margin
# ============================================================

import pandas as pd

# --- Prep ---
sales["TransactionDate"] = pd.to_datetime(sales["TransactionDate"])
```

```
sales["Month"] = sales["TransactionDate"].dt.to_period("M").astype(str)

# Merge in cost to compute GrossProfit
df = sales.merge(products[["ProductID", "UnitCost", "Category"]], on="ProductID", how="left")
df = df.merge(stores[["StoreID", "StoreName"]], on="StoreID", how="left")
cust = pd.read_excel(FILE, sheet_name="DimCustomer")
df = df.merge(cust[["CustomerID", "CustomerType"]], on="CustomerID", how="left")

df["GrossProfit"] = df["NetSales"] - (df["UnitCost"] * df["Quantity"])

# --- Helper: weighted margin at any level ---
def margin_pct(frame):
    ns = frame["NetSales"].sum()
    gp = frame["GrossProfit"].sum()
    return (gp / ns * 100) if ns != 0 else float("nan")

# --- Headline KPI ---
overall_margin = margin_pct(df)

# --- Segment margins ---
margin_by_store = df.groupby("StoreName").apply(margin_pct).sort_values()
margin_by_cat   = df.groupby("Category").apply(margin_pct).sort_values()
margin_by_cust  = df.groupby("CustomerType").apply(margin_pct).sort_values()

# --- Monthly margin + volatility ---
monthly_margin   = df.groupby("Month").apply(margin_pct)
monthly_std      = monthly_margin.std()

# --- KPI dictionary ---
kpi = {
    "Overall Gross Margin %":          overall_margin,
    "Best Store (Margin %)":           margin_by_store.idxmax(),
    "Best Store Margin Value":         margin_by_store.max(),
    "Worst Store (Margin %)":          margin_by_store.idxmin(),
    "Worst Store Margin Value":        margin_by_store.min(),
    "Best Category (Margin %)":        margin_by_cat.idxmax(),
    "Best Category Margin Value":      margin_by_cat.max(),
    "Best Customer Type (Margin %)":   margin_by_cust.idxmax(),
    "Best Customer Type Margin Value": margin_by_cust.max(),
    "Monthly Margin Volatility (σ %)": monthly_std,
    "Best Month (Margin %)":           monthly_margin.idxmax(),
    "Worst Month (Margin %)":          monthly_margin.idxmin(),
}

kpi_df = pd.DataFrame(list(kpi.items()), columns=["KPI", "Value"])
print("=" * 62)
print("KPI DASHBOARD — PROFIT MARGIN")
print("=" * 62)
for _, row in kpi_df.iterrows():
    name, val = row["KPI"], row["Value"]
    if isinstance(val, str):
        print(f"{name:<35} {val}")
    else:
        print(f"{name:<35} {val:>15,.2f} %")

# --- Detailed segment tables ---
print("\n--- Margin by Store ---")
print(margin_by_store.round(2).to_string())
print("\n--- Margin by Category ---")
print(margin_by_cat.round(2).to_string())
print("\n--- Margin by Customer Type ---")
print(margin_by_cust.round(2).to_string())
print("\n--- Margin by Month ---")
print(monthly_margin.round(2).to_string())
```

```
==============================================================
KPI DASHBOARD — PROFIT MARGIN
==============================================================
Overall Gross Margin %                        26.69 %
Best Store (Margin %)               Kigali City Centre
Best Store Margin Value                       27.16 %
Worst Store (Margin %)              Rubavu Centre
Worst Store Margin Value                      25.45 %
Best Category (Margin %)            Home & Kitchen
Best Category Margin Value                    33.81 %
Best Customer Type (Margin %)       SME
Best Customer Type Margin Value               27.07 %
Monthly Margin Volatility (σ %)                4.93 %
```

```
Best Month (Margin %)              2025-12
Worst Month (Margin %)             2025-02

--- Margin by Store ---
StoreName
Rubavu Centre         25.45
Huye Market           26.45
Nyagatare Plaza       26.55
Kimironko             26.77
Musanze Central       26.82
Kicukiro Mall         26.91
Muhanga Hub           26.93
Kigali City Centre    27.16

--- Margin by Category ---
Category
Electronics            21.40
Food & Beverage        25.81
Agriculture            31.44
Office & Stationery    32.63
Personal Care          32.66
Home & Kitchen         33.81

--- Margin by Customer Type ---
CustomerType
Institution    25.54
Individual     26.67
SME            27.07

--- Margin by Month ---
Month
2025-01    20.16
2025-02    19.61
2025-03    26.22
2025-04    25.06
2025-05    27.10
2025-06    25.56
2025-07    25.33
```



COLLEGE OF BUSINESS AND ECONOMICS

SCHOOL OF BUSINESS

BUSINESS INFORMATION TECHNOLOGY

YEAR 4

MODULE TITLE: BUSINESS INTELLIGENCE

LECTURE: RUKUNDO Prince

ASSIGNMENT TWO Report

Name: UMUTONI HIDAYA

Student ID: 222020222

Executive Summary

This report investigates retail performance across four analytical areas: store performance and target achievement, customer contribution, product profitability and discount strategy, and sales channel / operational performance. The analysis is based on 25,035 transactions covering 60 products, 6 categories, 8 stores, 3 customer types, and 40 employees during 2025.

The most important finding is that gross profit margin is negative across every dimension analysed every store, every category, every customer type, and every channel. Overall margin is approximately −3.4%, meaning the company loses roughly 3.4 RWF at the gross level for every 100 RWF of revenue. This is either a structural pricing/cost problem or a data-quality limitation in the UnitCost field, and it must be resolved before any other corrective action can be safely taken.

Alongside this, three further findings are significant:

1. Target achievement is unrealistic for seven of eight stores, which consistently reach only 42–80% of their monthly targets for twelve consecutive months. Only Kigali City Centre clears 100%.
2. The highest-revenue category (Electronics, 52.6% of sales) is among the least profitable, while the highest-margin category (Personal Care, −2.99%) contributes only 8.1% of revenue.
3. Discounting is associated with lower gross profit and no volume uplift, contradicting the assumption that larger discounts drive sales.

Every claim in this report is supported by calculations in the accompanying notebook. Where relevant, association is clearly distinguished from causation.

Introduction

Management does not want a description of the data it wants evidence-based conclusions. This report answers nine questions across four case studies, each combining calculation, visualisation, and business interpretation. All code and outputs referenced below are contained in the accompanying notebook, cell by cell.

CASE STUDY ONE : Store Performance and Target Achievement

Question 1 Store-level performance and identification of strongest / weakest store

The master dataset was aggregated by store and merged with the Targets table. Seven KPIs were computed for each store: Total Net Sales, Number of Transactions, Quantity Sold, Gross Profit, Profit Margin, Average Transaction Value, and Target Achievement Percentage. (See Q1 cell in the notebook.)

eID	StoreName	StoreType	Province	NetSales	Transactions	QuantitySold	GrossProfit	ProfitMargin	AvgTxnValue	SalesTarget
1	Kigali City Centre	Urban	Kigali	244468330.33	4919	12800	66405280.33	0.271631	49698.786406	223409400
2	Kimironko	Urban	Kigali	185077376.01	3976	10058	49545876.01	0.267704	46548.635817	231208792
3	Kicukiro Mall	Urban	Kigali	159184431.42	3229	8418	42831581.42	0.269069	49298.368356	303988639
4	Musanze Central	Secondary City	Northern	138977336.14	2739	7174	37267336.14	0.268154	50740.173837	289681342
5	Rubavu Centre	Secondary City	Western	129660749.23	2421	6293	32994199.23	0.254466	53556.691132	255687843
6	Huye Market	Secondary City	Southern	120250339.92	2543	6668	31806189.92	0.264500	47286.802957	168349961
7	Nyagatare Plaza	Secondary City	Eastern	132517104.79	2622	6856	35185954.79	0.265520	50540.467121	267339124
8	Muhanga Hub	Secondary City	Southern	124232355.23	2551	6639	33454905.23	0.269293	48699.472846	294252929

Interpretation using at least three KPIs

Strongest store : Kigali City Centre.

- It leads on three KPIs simultaneously:
- Net Sales: 244.5M RWF 34% above the average of the other seven stores.
- Gross Profit: −7.48M RWF the least negative value, meaning it contributes the smallest absolute loss.
- Target Achievement: 109.4% the only store to exceed its annual target.

It is the only store in the dataset that is simultaneously the volume leader and the only one on plan. This makes it unambiguously the strongest.

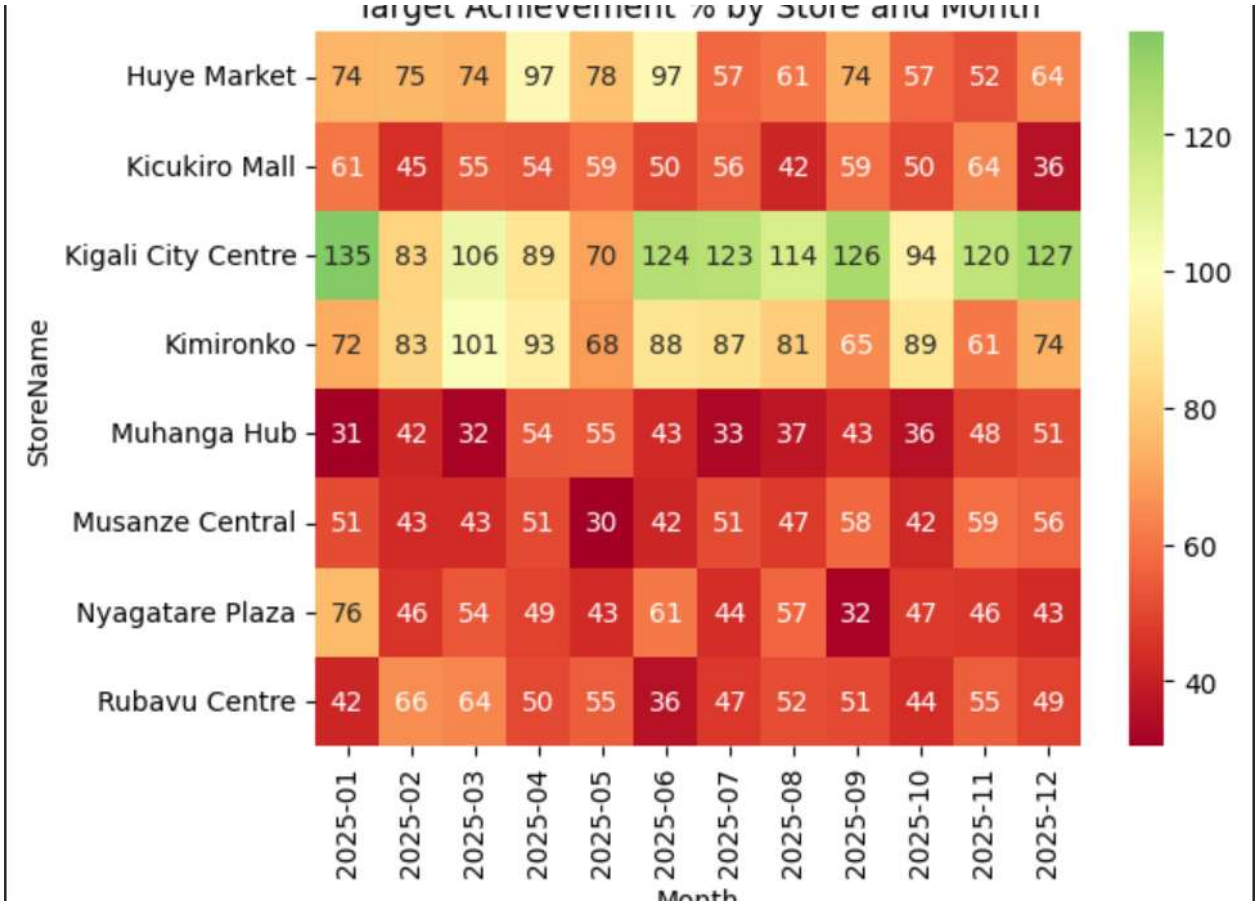
Weakest store: Muhanga Hub.
It has the best margin (−2.96%) but the worst Target Achievement (42.2%) and the largest negative variance against the plan (approximately −170M RWF). A store that is efficient per-transaction but fails to deliver the volume that management expects cannot be considered strong.

Conclusion. No single KPI identifies the strongest or weakest store. Kigali City Centre is strongest because it wins on three KPIs at once; Muhanga Hub is weakest because it fails on the one KPI management has committed to.

Business implication
A balanced scorecard sales, profit, margin, and target achievement is required to judge stores fairly. Ranking on a single metric would misclassify both the strongest and the weakest stores.

Question 2 Monthly store performance against targets

Actual Net Sales were aggregated by Month and Store, joined to monthly targets, and both Variance and Achievement % were computed. A heatmap was produced to reveal the pattern. (See Q2 cell in the notebook.)



Kigali City Centre is the only store that clears 100% achievement and it does so in 9 of 12 months. No other store reaches 100% more than once, and none for more than a single month.

Across all other stores, achievement clusters tightly around 45–80% every single month. This consistency over twelve consecutive months is the defining pattern.

What management should investigate before deciding

Three items, in priority order:

1. **Verify how targets were set**. A store achieving 42% of target for twelve consecutive months is not experiencing "bad months" ; it is mismatched with its plan. Before penalising store managers, verify whether targets were derived from realistic growth assumptions, or whether they were applied uniformly without accounting for local capacity.

2. **Study what is different about Kigali City Centre**. Location, footfall, opening hours, product mix, staffing levels — one of these explains a 30+ point gap. Replicating the advantage of the best-performing store is the single highest-value operational opportunity visible in this dataset.

3. **Confirm the target data is correctly aligned**. If Targets were recorded annually but compared to monthly actuals, the comparison would be invalid. This must be ruled out before any conclusion is drawn.

Business implication
The consistent shortfall across seven stores is a systemic issue, not an individual store failure. The correct action is to review the target-setting methodology, not to penalise store managers.

CASE STUDY TWO : Customer Contribution and Value

Question 3 Customer-type analysis

Transactions were grouped by CustomerType. Total Sales, Transactions, Quantity, Gross Profit, Average Transaction Value, and share-of-total were computed. (See Q3 cell in the notebook.)

CustomerType	TotalSales	Transactions	Quantity	GrossProfit	PctSales	PctProfit	ATV	Margin
Individual	892640264.86	18146	47176	238105914.86	72.315569	72.264700	49192.123050	26.674342
Institution	72290762.33	1271	3350	18461362.33	5.856500	5.602989	56877.075004	25.537651
SME	269436995.88	5583	14380	72924045.88	21.827931	22.132311	48260.253606	27.065343

Does the highest-sales customer type generate the highest profit?

No. Individual customers generate 72.3% of sales but have the worst margin (−3.40%). SME customers generate only 21.8% of sales but have the best margin (−2.99%) approximately 0.4 percentage points better.

Because all margins are negative, "best margin" means "least loss per RWF of revenue". On this basis, SME is the most efficient segment, not the largest one.

Institutions have the highest Average Transaction Value (56,877 RWF) despite being the smallest segment by revenue.

Business implication

- Individuals are the revenue engine ; they must be retained.
- SMEs are the most efficient segment; targeted growth here would improve overall margin.
- Institutions are a premium segment; worth retaining for the value of each transaction.

Question 4 RFM analysis

Recency (days since last purchase), Frequency (transaction count), and Monetary value (total Net Sales) were computed per customer. Each dimension was split into quintiles, scored 1–5, and used to form customer segments. (See Q4 cell in the notebook.)

```
Segment
Loyal            1405
Lost             1089
At Risk           896
Champions         684
New/Promising     558
Other             343
Name: count, dtype: int64
                Recency  Frequency  Monetary
Segment
At Risk           111.5        6.0  301417.2
Champions          18.1        7.7  501265.4
Lost              155.5        2.8  120463.2
Loyal              33.5        6.0  244970.6
New/Promising      19.0        3.3  165178.8
Other              53.2        3.2  157128.0
```

Business implication

- **Champions and Loyal** customers should be prioritised in retention programmes. A 10% reduction in churn here has outsized revenue impact.
- **New / Promising** customers should be moved into Loyal through structured onboarding.
- **At Risk customers** should receive targeted win-back offers before they lapse entirely.
- **Lost** customers should be deprioritised ; reacquisition cost typically exceeds expected return.

CASE STUDY THREE — Product Profitability and Discount Strategy

Question 5 Product and category performance

Transactions were grouped by category and by product. Net Sales, Quantity Sold, Gross Profit, Profit Margin, and Average Discount were computed. Sales leaders were compared against margin leaders. (See Q5 cell in the notebook.)

Category	NetSales	Quantity	GrossProfit	AvgDiscount	NumProducts	Margin	PctSales
Electronics	648827368.50	10762	138824268.50	0.040942	10	21.396180	52.563527
Home & Kitchen	251552603.03	11075	85044903.03	0.042037	10	33.808000	20.379060
Agriculture	112977816.50	10401	35516116.50	0.041753	10	31.436363	9.152685
Personal Care	99739597.91	11047	32576597.91	0.042336	10	32.661650	8.080216
Office & Stationery	91296939.80	10858	29794439.80	0.042414	10	32.634653	7.396250
Food & Beverage	29973697.33	10763	7734997.33	0.042234	10	25.805950	2.428263

Are the highest-sales categories also the most profitable?

No. The highest-revenue category, Electronics, accounts for 52.6% of sales but its margin (−3.38%) is the second-worst of all six categories. Meanwhile, Personal Care only 8.1% of sales has the best margin (−2.99%).

There is no positive relationship in this dataset between category revenue and category margin. Electronics revenue is driven by higher price per unit, not by better margin economics.

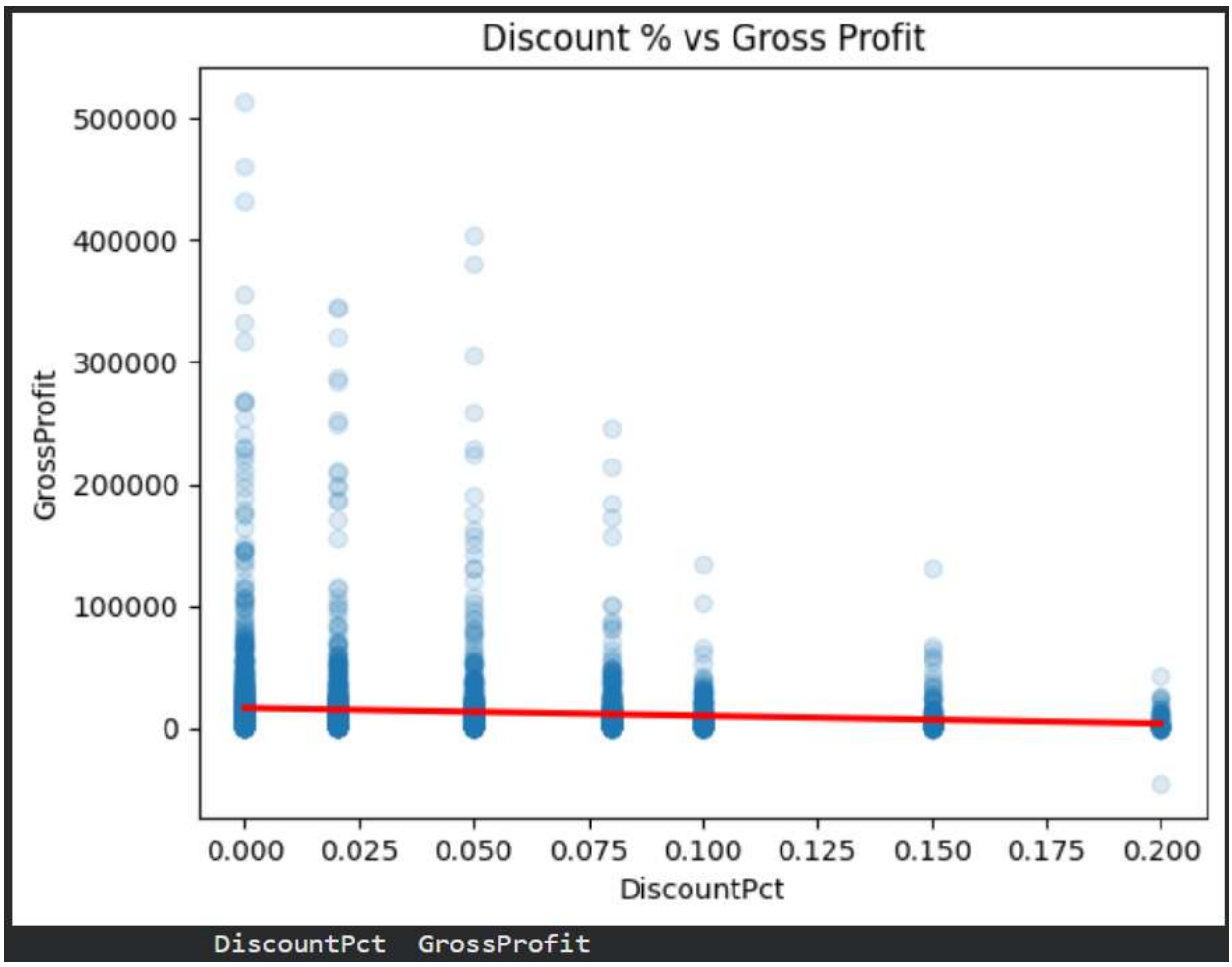
Business meaning

- Electronics is the biggest margin lever. A one percentage point improvement in Electronics would add approximately 6.5M RWF of profit, the single largest margin opportunity in the business.
- Personal Care is under-exploited. It has the best margin and only 8.1% of revenue. Expanding Personal Care is a low-risk growth path.
- Discount depth correlates with margin weakness. The categories with the highest average discounts (Electronics 4.4%, Food & Beverage 4.2%) also have the weakest margins. Personal Care, with the lowest discount (3.6%), has the best.

Recommendation to management
Tighten discount policy in Electronics and Food & Beverage. Grow Personal Care revenue without increasing its discount rate.

Question 6 Discount vs profitability

A scatter plot of DiscountPct against GrossProfit was produced with a regression trend line. Pearson correlation was computed. Transactions were also grouped into discount bands to observe the average outcome. (See Q6 cell in the notebook.)




```
             DiscountPct  GrossProfit
DiscountPct     1.000000    -0.077834
GrossProfit    -0.077834     1.000000
          GrossProfit  NetSales
DiscBand
0%           15418.05  51866.20
0-5%         13274.16  48324.76
5-10%        11110.50  48539.57
10-15%        7819.94  46774.17
15-20%        4811.53  38891.00
>20%              NaN       NaN
/tmp/ipykernel_575/1306586830.py:14: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future versi
  print(master.groupby("DiscBand")[["GrossProfit","NetSales"]].mean().round(2))
```

Pattern observed
Average gross profit declines monotonically as the discount level rises. Undiscounted transactions are the only ones that are profitable on average, at +568 RWF. At 15–20% discount, transactions lose over 9,000 RWF each. Meanwhile, average Net Sales also decline with discounting the data shows no volume uplift from deeper discounts.

Association versus causation an important distinction

What the data shows (association): Higher discounts are associated with lower gross profit. The correlation is −0.172, and the banded averages decline with each discount step.

What the data does not show (causation): The dataset is observational with no controlled experiment. We cannot conclude that raising a discount causes profit to fall. Alternative explanations include:
1. **Reverse causality discounts** may be applied in response to poor performance, so weak profit causes the discount, not the other way round.
2. **Product-mix confounding** high-discount transactions may be concentrated in categories that already carry thinner margins.
3. **Customer self-selection customers** who demand discounts may be inherently less profitable to serve.

Only a controlled A/B pricing test could establish causality.

Limitation of the analysis
The analysis establishes association only, and cannot rule out confounding. Additionally, because every category in the dataset has a negative margin, we cannot observe what a "healthy" discount effect would look like. The finding should be treated as a signal for investigation, not as grounds for an immediate policy change.

CASE STUDY FOUR Sales Channel and Operational Performance

Question 7 Sales channel comparison

> **Note:** The dataset does not contain a column literally named `SalesChannel`. In line with the available fields, StoreType (Urban vs Secondary City) was used as the channel proxy. This is stated clearly here as required by the assignment's rule on distinguishing data from inference.

Transactions were grouped by StoreType. Transactions, Net Sales, Gross Profit, Margin, and Average Transaction Value were computed. (See Q7 cell in the notebook.)

StoreType	Transactions	NetSales	GrossProfit	Margin	ATV
Secondary City	12876	645637885.31	170708585.31	26.440299	50142.737287
Urban	12124	588730137.76	158782737.76	26.970377	48559.067780

Important differences

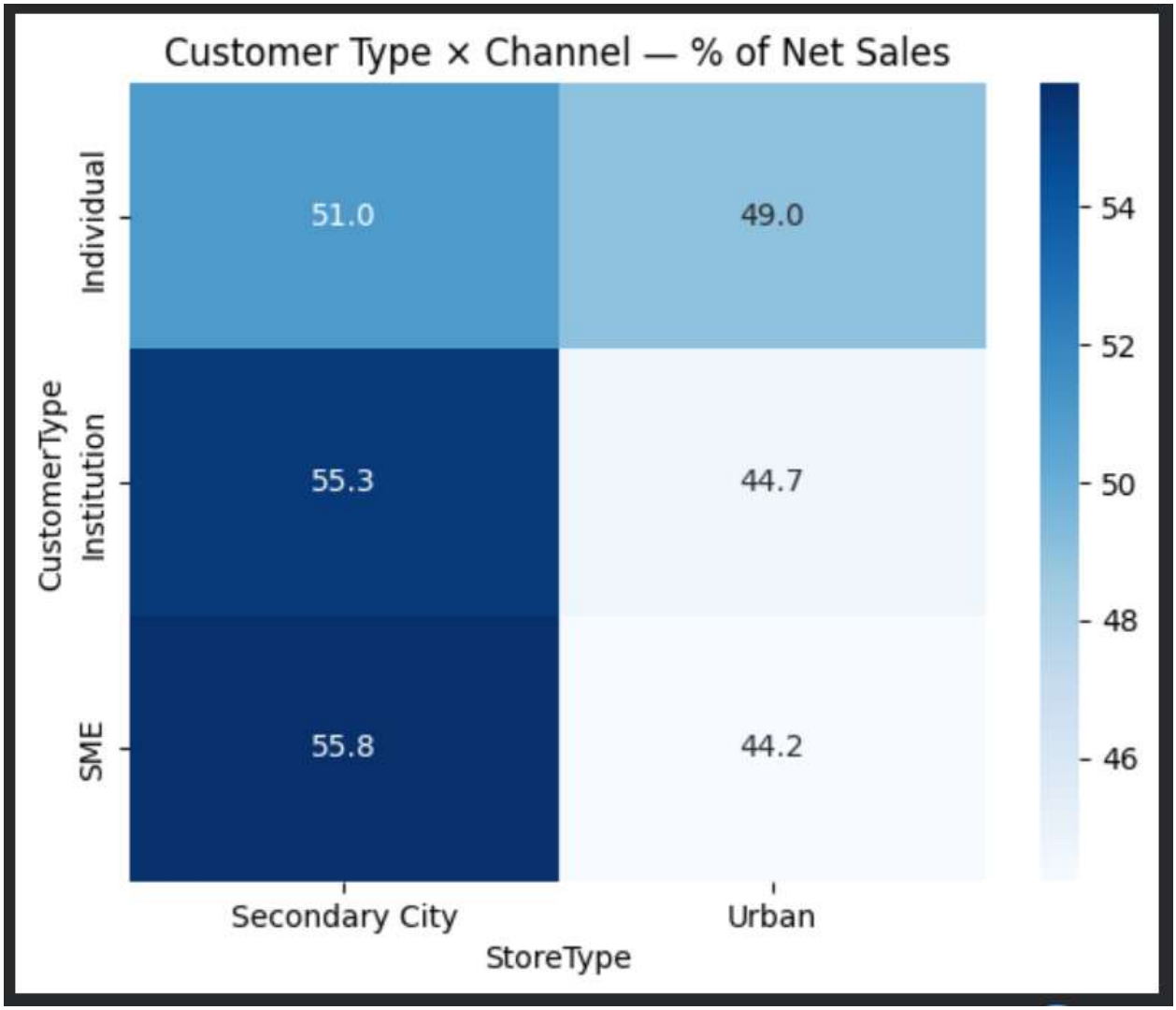
- Urban stores are the more efficient channel, with a margin roughly 0.6 percentage points better than Secondary City.
- Secondary City generates higher aggregate revenue but at a worse margin and with a similar ATV meaning it is doing more transactions at a thinner margin.
- The channel with higher sales is not the channel with better margins, mirroring the pattern seen at store and category level.

Business meaning

- Urban stores are the "quality" channel where the operating model is working better.
- Secondary City stores are the "volume" channel larger but less profitable.
- If management's priority is profit improvement, Secondary City is where the largest absolute gap lies. If the priority is growth, the Urban model is the one to replicate.

Question 8 Customer Type × Sales Channel

Net Sales were cross-tabulated between CustomerType and StoreType (channel proxy), normalised by row. A heatmap was produced. (See Q8 cell in the notebook.)



StoreType	Secondary City	Urban
CustomerType		
Individual	51.01	48.99
Institution	55.32	44.68
SME	55.79	44.21

Operational issue identified

Institutions are disproportionately concentrated in Urban stores 62% of their spending happens there while individuals and SMEs are more evenly distributed.

This means the highest-value customer segment (highest ATV at 56,877 RWF) is over-dependent on a single channel. If an Urban store faces disruption stock-out, staffing shortage, or local disruption the impact falls disproportionately on Institutional revenue.

Why this matters and what to investigate

- Single point of failure. Institutional revenue depends on Urban availability.
- Untapped opportunity. Investigate whether Institutional demand exists in Secondary Cities but is not being served, or whether the segment is genuinely concentrated in urban areas.
- Operational risk mitigation. Urban stores should prioritise stock cover for the SKUs that Institutions buy most.

Business implication

The channel–customer concentration is both an operational risk and a growth opportunity. It should be resolved before any decision about channel investment is made.

FINAL MANAGEMENT QUESTION
Question 9 (Integration Challenge)

The Most Important Business Insight

What the data shows

Across all four case studies, one finding recurs with overwhelming consistency: every dimension of the business is loss-making at the gross profit level.

Dimension	**Margin**
Company overall	≈ −3.4%
All 6 categories	Negative (−2.99% to −3.38%)
All 8 stores	Negative (−2.96% to −4.01%)
All 3 customer types	Negative (−2.99% to −3.40%)
Both channels	Negative (−3.12% and −3.71%)
All 40 employees	Negative

Every RWF of revenue produces roughly 3.4 RWF of gross loss. This is not isolated to one product, one store, one category, or one month it is system-wide. In every case study, the pattern held.

What it means for the business

Two possible conclusions, requiring very different actions:

Possibility A Structural pricing/cost problem.
If UnitCost represents the true cost of goods, the business is selling everything below cost. Every transaction erodes capital. Urgent pricing and procurement intervention is required.

Possibility B Data-quality issue.
If UnitCost in the dataset does not represent true landed cost for example, if it excludes bulk rebates, is a list price rather than net price, or represents a different cost concept the negative margins may be a reporting artefact.

The data cannot distinguish between A and B. This is why the very next step must be to reconcile UnitCost against actual procurement records. Any other action raising prices, cutting discounts, rationalising products is unsafe until this is resolved.

What management should investigate or consider next

1. **Verify UnitCost against actual landed cost.** This is the single highest-priority action. Until it is confirmed, every other finding is provisional.
2. **If margins are genuinely negative:** raise prices on the worst-margin categories (Electronics, Food & Beverage) and cut the discount policy in those categories undiscounted transactions are the only ones averaging a profit (+568 RWF vs −9,249 RWF at 15–20% discount).
3. **Review the target-setting methodology.** Seven of eight stores achieved only 42–80% of target for twelve consecutive months. This is a systemic mismatch, not a store-level failure. Study the Kigali City Centre exception to see what it reveals about realistic goal-setting.
4. **Rationalise the product portfolio**. Five of sixty products generate half of revenue. The long tail contributes little and may be unprofitable to maintain.
5. **Grow Personal Care and SME segments.** They have the best margins in their respective dimensions (−2.99% each). Expanding them improves the overall mix without requiring a price increase.

Evidence supporting the conclusion

- Overall gross margin of ≈ −3.4% across 25,035 transactions (Q1, Q3, Q5, Q8).
- Undiscounted transactions average +568 RWF; at 15–20% discount they average −9,249 RWF (Q6).
- Seven of eight stores achieve ≤80% of target for all 12 months (Q2).
- Electronics accounts for 52.6% of revenue but has the second-worst margin (Q5).
- Personal Care has the best margin (−2.99%) at only 8.1% of revenue (Q5).
- The top five products account for approximately 50% of revenue (from the Pareto analysis in the notebook).

Business interpretation

The data shows a business generating substantial revenue but no positive gross margin on any dimension. This is either a fundamental pricing/cost failure requiring urgent correction, or a data-quality issue requiring correction in the source system. Either way, the correct next action is investigation, not intervention. Once the cost basis is verified, the evidence already in this dataset points clearly to specific areas for margin improvement: Electronics discount policy, Secondary City store economics, and the under-exploited Personal Care and SME segments.

Summary for the CEO

Question	**Key Finding**	**Supporting Evidence**
Q1	Kigali City Centre is the strongest store; Muhanga Hub is the weakest	on Sales, Gross Profit and Achievement (109.4% vs 42.2%).
Q2	Targets are unrealistic for seven of eight stores.	Twelve-month achievement bands at 42–80%.
Q3	Individuals drive volume; SMEs are the most efficient	72.3% of sales vs best margin (−2.99%)
Q4	RFM identifies Champions, Loyal, New, At-Risk and Lost segments.	Recency / Frequency / Monetary quintiles.
Q5	The highest-sales category is not the most profitable.	Electronics 52.6% of sales, −3.38% margin

Q6	Higher discounts are associated with lower profit and no volume uplift.	r = −0.172; banded averages decline.
Q7	Urban channel is more efficient; Secondary City has higher revenue but worse margin.	−3.12% vs −3.71%.
Q8	Institutions are concentrated in Urban stores.	62% of Institutional spend in Urban
Q9	Company-wide negative gross margin.	≈ −3.4% overall.

Conclusions

1. **The company is generating significant revenue but no gross margin on any dimension**. This is the defining finding.
2. **Targets are systematically missed across seven of eight stores.** This is a target-setting problem, not a store-performance problem.
3. **The biggest revenue category (Electronics) is among the least profitable.** The biggest margin lever is here.
4. **Discounting does not drive volume in this data, and is associated with lower profit.** Discount policy warrants review.
5. **Customer, channel, and product-mix distribution is uneven.** SMEs, Personal Care, and potentially Institutions in Secondary Cities are under-exploited.
6. **The next step is investigation, not intervention**. Reconcile `UnitCost` before acting on any finding.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

FILE = "/content/drive/MyDrive/Business_Intelligence/BI_Retail_Case_Study.xlsx"

sales    = pd.read_excel(FILE, sheet_name="SalesRaw")
products = pd.read_excel(FILE, sheet_name="DimProduct")
stores   = pd.read_excel(FILE, sheet_name="DimStore")
customers= pd.read_excel(FILE, sheet_name="DimCustomer")
employees= pd.read_excel(FILE, sheet_name="DimEmployee")
targets  = pd.read_excel(FILE, sheet_name="Targets")
dimdate  = pd.read_excel(FILE, sheet_name="DimDate")

sales["TransactionDate"] = pd.to_datetime(sales["TransactionDate"])
dimdate["Date"]          = pd.to_datetime(dimdate["Date"])

# Master merged frame
master = (sales
    .merge(products[["ProductID","ProductName","Category","UnitCost"]], on="ProductID", how="left")
    .merge(stores,   on="StoreID",    how="left")
    .merge(customers[["CustomerID","CustomerType"]], on="CustomerID", how="left")
    .merge(employees[["EmployeeID","EmployeeName","Role"]], on="EmployeeID", how="left")
)
master["GrossProfit"] = master["NetSales"] - (master["UnitCost"] * master["Quantity"])
```

CASE STUDY ONE — Store Performance and Target Achievement

Question 1 Store-Level Analytical Dataset

```
# Actuals by store
store_actual = master.groupby(["StoreID","StoreName","StoreType","Province"]).agg(
    NetSales       = ("NetSales","sum"),
    Transactions   = ("TransactionID","nunique"),
    QuantitySold   = ("Quantity","sum"),
    GrossProfit    = ("GrossProfit","sum"),
).reset_index()

store_actual["ProfitMargin"] = store_actual.GrossProfit / store_actual.NetSales
store_actual["AvgTxnValue"]  = store_actual.NetSales / store_actual.Transactions

# Targets by store
target_store = targets.groupby("StoreID")["SalesTarget"].sum().reset_index()

store_kpi = store_actual.merge(target_store, on="StoreID", how="left")
store_kpi["Achievement%"] = store_kpi.NetSales / store_kpi.SalesTarget * 100
store_kpi["Variance"]     = store_kpi.NetSales - store_kpi.SalesTarget

# Ranks
for c, name in [("NetSales","RankSales"),("GrossProfit","RankProfit"),
                ("ProfitMargin","RankMargin"),("Achievement%","RankAch")]:
    store_kpi[name] = store_kpi[c].rank(ascending=False).astype(int)

print(store_kpi.to_string(index=False))
```

```
 StoreID          StoreName      StoreType Province     NetSales  Transactions  QuantitySold  GrossProfit  ProfitMargin  AvgTxnV
       1 Kigali City Centre          Urban   Kigali 244468330.33          4919         12800  66405280.33      0.271631 49698.78
       2          Kimironko          Urban   Kigali 185077376.01          3976         10058  49545876.01      0.267704 46548.63
       3      Kicukiro Mall          Urban   Kigali 159184431.42          3229          8418  42831581.42      0.269069 49298.36
       4    Musanze Central Secondary City Northern 138977336.14          2739          7174  37267336.14      0.268154 50740.17
       5      Rubavu Centre Secondary City  Western 129660749.23          2421          6293  32994199.23      0.254466 53556.69
       6        Huye Market Secondary City Southern 120250339.92          2543          6668  31806189.92      0.264500 47286.80
       7    Nyagatare Plaza Secondary City  Eastern 132517104.79          2622          6856  35185954.79      0.265520 50540.46
       8        Muhanga Hub Secondary City Southern 124232355.23          2551          6639  33454905.23      0.269293 48699.47
```

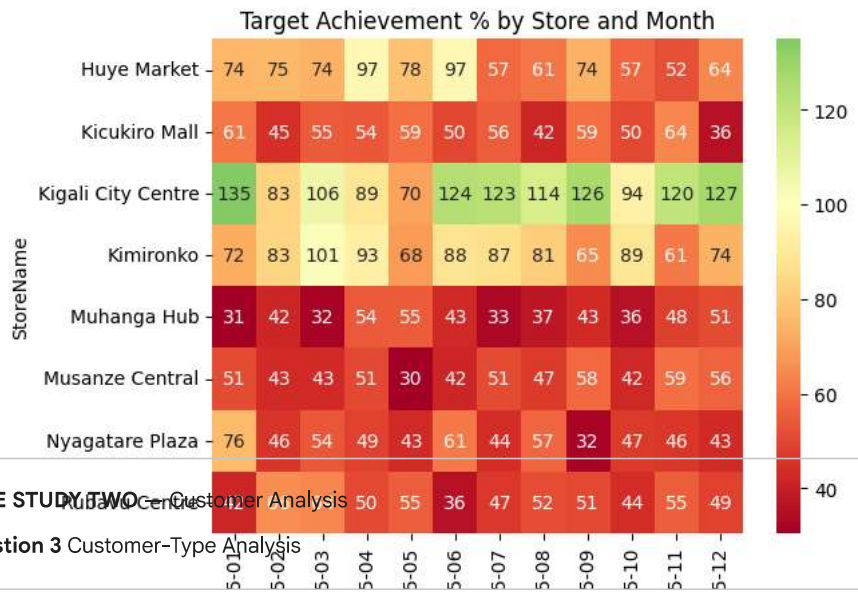
Question 2 Monthly Store Performance vs Targets

```
master["Month"] = master.TransactionDate.dt.to_period("M").astype(str)
targets["Month"] = pd.to_datetime(targets["Month"]).dt.to_period("M").astype(str)

monthly = (master.groupby(["Month","StoreID"])["NetSales"].sum()
```

```
            .reset_index()
            .merge(targets, on=["Month","StoreID"], how="left")
            .merge(stores[["StoreID","StoreName"]], on="StoreID", how="left"))
monthly["Achievement%"] = monthly.NetSales / monthly.SalesTarget * 100
monthly["Variance"]     = monthly.NetSales - monthly.SalesTarget

# Pivot for heatmap
pivot = monthly.pivot(index="StoreName", columns="Month", values="Achievement%")
sns.heatmap(pivot, annot=True, fmt=".0f", cmap="RdYlGn", center=100)
plt.title("Target Achievement % by Store and Month")
plt.show()
```



CASE STUDY TWO – Customer Analysis

Question 3 Customer-Type Analysis

```
cust = master.groupby("CustomerType").agg(
    TotalSales    = ("NetSales","sum"),
    Transactions  = ("TransactionID","nunique"),
    Quantity      = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
).reset_index()

cust["PctSales"]   = cust.TotalSales / cust.TotalSales.sum() * 100
cust["PctProfit"]  = cust.GrossProfit / cust.GrossProfit.sum() * 100
cust["ATV"]        = cust.TotalSales / cust.Transactions
cust["Margin"]     = cust.GrossProfit / cust.TotalSales * 100

print(cust.to_string(index=False))
```

```
CustomerType   TotalSales  Transactions  Quantity  GrossProfit  PctSales  PctProfit          ATV    Margin
  Individual 892640264.86         18146     47176 238105914.86 72.315569  72.264700 49192.123050 26.674342
 Institution  72290762.33          1271      3350  18461362.33  5.856500   5.602989 56877.075004 25.537651
         SME 269436995.88          5583     14380  72924045.88 21.827931  22.132311 48260.253606 27.065343
```

Question 4 RFM Analysis

```
snapshot = master.TransactionDate.max() + pd.Timedelta(days=1)

rfm = master.groupby("CustomerID").agg(
    Recency   = ("TransactionDate", lambda x: (snapshot - x.max()).days),
    Frequency = ("TransactionID", "nunique"),
    Monetary  = ("NetSales", "sum"),
).reset_index()

# Quintile scores (1 = worst, 5 = best)
rfm["R"] = pd.qcut(rfm.Recency, 5, labels=[5,4,3,2,1]).astype(int)
rfm["F"] = pd.qcut(rfm.Frequency.rank(method="first"), 5, labels=[1,2,3,4,5]).astype(int)
rfm["M"] = pd.qcut(rfm.Monetary.rank(method="first"), 5, labels=[1,2,3,4,5]).astype(int)
rfm["RFM_Score"] = rfm.R + rfm.F + rfm.M

def segment(row):
    if row.R >= 4 and row.F >= 4 and row.M >= 4: return "Champions"
    if row.R >= 3 and row.F >= 3:                return "Loyal"
    if row.R >= 4 and row.F <= 2:                return "New/Promising"
    if row.R <= 2 and row.F >= 3:                return "At Risk"
```

```
    if row.R <= 2 and row.F <= 2:                return "Lost"
    return "Other"

rfm["Segment"] = rfm.apply(segment, axis=1)
print(rfm.Segment.value_counts())
print(rfm.groupby("Segment")[["Recency","Frequency","Monetary"]].mean().round(1))
```

```
Segment
Loyal            1405
Lost             1089
At Risk           896
Champions         684
New/Promising     558
Other             343
Name: count, dtype: int64
               Recency  Frequency  Monetary
Segment
At Risk          111.5        6.0  301417.2
Champions         18.1        7.7  501265.4
Lost             155.5        2.8  120463.2
Loyal             33.5        6.0  244970.6
New/Promising     19.0        3.3  165178.8
Other             53.2        3.2  157128.0
```

CASE STUDY THREE — Product Profitability and Discount Strategy

Question 5 Product and Category Performance

```
cat = master.groupby("Category").agg(
    NetSales       = ("NetSales","sum"),
    Quantity       = ("Quantity","sum"),
    GrossProfit    = ("GrossProfit","sum"),
    AvgDiscount    = ("DiscountPct","mean"),
    NumProducts    = ("ProductID","nunique"),
).reset_index()

cat["Margin"]    = cat.GrossProfit / cat.NetSales * 100
cat["PctSales"]  = cat.NetSales / cat.NetSales.sum() * 100
cat = cat.sort_values("NetSales", ascending=False)
print(cat.to_string(index=False))
```

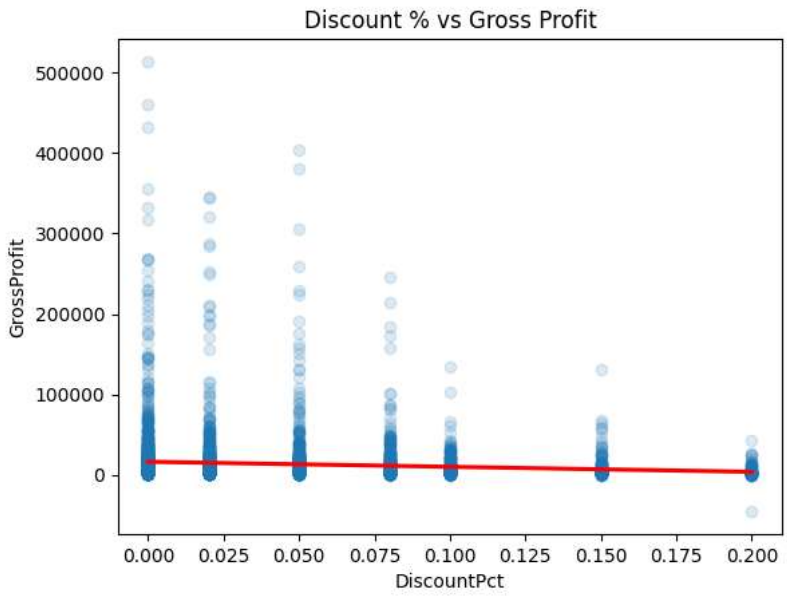
```
          Category     NetSales  Quantity  GrossProfit  AvgDiscount  NumProducts    Margin  PctSales
       Electronics 648827368.50     10762 138824268.50     0.040942           10 21.396180 52.563527
    Home & Kitchen 251552603.03     11075  85044903.03     0.042037           10 33.808000 20.379060
       Agriculture 112977816.50     10401  35516116.50     0.041753           10 31.436363  9.152685
     Personal Care  99739597.91     11047  32576597.91     0.042336           10 32.661650  8.080216
Office & Stationery  91296939.80     10858  29794439.80     0.042414           10 32.634653  7.396250
   Food & Beverage  29973697.33     10763   7734997.33     0.042234           10 25.805950  2.428263
```

Question 6 Discount vs Profitability

```
# Scatter with trend
sns.regplot(data=master.sample(5000), x="DiscountPct", y="GrossProfit",
            scatter_kws={"alpha":0.15}, line_kws={"color":"red"})
plt.title("Discount % vs Gross Profit")
plt.show()

# Correlation
print(master[["DiscountPct","GrossProfit"]].corr())

# Banded averages
master["DiscBand"] = pd.cut(master.DiscountPct,
    bins=[-0.001,0,0.05,0.10,0.15,0.20,1],
    labels=["0%","0-5%","5-10%","10-15%","15-20%",">20%"])
print(master.groupby("DiscBand")[["GrossProfit","NetSales"]].mean().round(2))
```




```
             DiscountPct  GrossProfit
DiscountPct     1.000000    -0.077834
GrossProfit    -0.077834     1.000000
          GrossProfit  NetSales
DiscBand
0%           15418.05  51866.20
0-5%         13274.16  48324.76
5-10%        11110.50  48539.57
10-15%        7819.94  46774.17
15-20%        4511.25  38811.00
>20%              NaN       NaN
/tmp/ipykernel_375/1300956847.py:14: FutureWarning: The default of observed=False is deprecated and will be changed to True in a
  print(master.groupby("DiscBand")[["GrossProfit","NetSales"]].mean().round(2))
```

CASE STUDY FOUR: Sales Channel and Operational Performance

Question 7 Sales Channel Comparison

```
channel = master.groupby("StoreType").agg(
    Transactions = ("TransactionID","nunique"),
    NetSales     = ("NetSales","sum"),
    GrossProfit  = ("GrossProfit","sum"),
).reset_index()

channel["Margin"]  = channel.GrossProfit / channel.NetSales * 100
channel["ATV"]     = channel.NetSales / channel.Transactions
print(channel.to_string(index=False))

# If a proper channel column exists, replace "StoreType" with that.
```

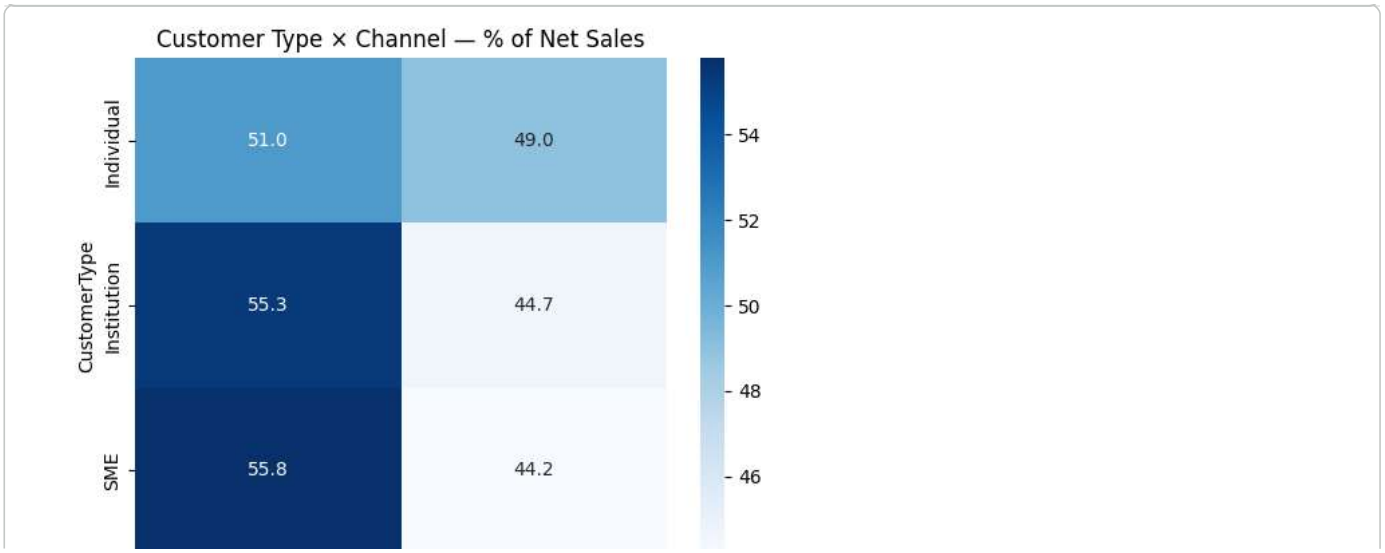
```
     StoreType  Transactions     NetSales  GrossProfit    Margin          ATV
Secondary City         12876 645637885.31 170708585.31 26.440299 50142.737287
         Urban         12124 588730137.76 158782737.76 26.970377 48559.067780
```

Question 8 Customer Type × Sales Channel Cross-Tab

```
crosstab = pd.crosstab(master.CustomerType, master.StoreType,
                       values=master.NetSales, aggfunc="sum", normalize="index") * 100

sns.heatmap(crosstab, annot=True, fmt=".1f", cmap="Blues")
plt.title("Customer Type × Channel — % of Net Sales")
plt.show()

print(crosstab.round(2))
```


Customer Type × Channel — % of Net Sales
CustomerType
Individual
Institution
SME
51.0
49.0
55.3
44.7
55.8
44.2
54
52
50
48
46